🧠 AI Evolution Program

Parte 04 — Redes neuronales y deep learning

Explica la transición desde el perceptrón a arquitecturas profundas. La especialización práctica completa vive en neural-network-training-labs.

1. 049 — Perceptrón y límites de separabilidad
2. 050 — MLP y backpropagation
3. 051 — Activaciones, inicialización y normalización
4. 052 — Optimizadores, regularización y schedulers
5. 053 — CNN y aprendizaje espacial
6. 054 — RNN, LSTM y secuencias
7. 055 — Atención y arquitectura Transformer
8. 056 — Graph Neural Networks
9. 057 — Aprendizaje por refuerzo profundo
10. 058 — Autoencoders, GAN y difusión
11. 059 — Transferencia, fine-tuning y destilación
12. 060 — Proyecto: modelo trazable de extremo a extremo

049 — Perceptrón y límites de separabilidad

Parte: 04 — Redes neuronales y deep learning
Nivel: intermedio-avanzado · **Horas estimadas:** 6
Laboratorio: `neural` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **perceptrón y límites de separabilidad** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar perceptrón y límites de separabilidad usando los conceptos `perceptrón`, `separabilidad`, `pesos`, `sesgo`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`perceptrón`, `separabilidad`, `pesos`, `sesgo`

🗺️ Ubicación en el mapa de la IA

El perceptrón (Rosenblatt, 1958) es la primera máquina de aprendizaje conexionista: en lugar de programar reglas simbólicas, ajusta pesos a partir de ejemplos. Su límite —solo resuelve problemas linealmente separables, como demostraron Minsky y Papert en 1969 con XOR— provocó el primer "invierno" de las redes neuronales y motiva directamente la clase siguiente: capas ocultas y backpropagation rompen esa barrera.

📖 Fundamentos

🧠 De la neurona biológica al modelo matemático

Una neurona artificial recibe entradas $x = (x_1, \ldots, x_n)$, las pondera con pesos $w = (w_1, \ldots, w_n)$, suma un sesgo b y aplica una función de activación. El perceptrón usa la función escalón:

$$z = w \cdot x + b = \Sigma_i \, w_i \cdot x_i + b$$
$$\hat{y} = 1 \quad \text{si } z \geq 0$$
$$\hat{y} = 0 \quad \text{si } z < 0$$

Geométricamente, `w · x + b = 0` define un hiperplano: una recta en 2D, un plano en 3D. El perceptrón clasifica según el lado del hiperplano en que cae cada punto. El sesgo b desplaza el hiperplano fuera del origen; sin él, la frontera pasaría siempre por (0, 0).

🔁 Regla de aprendizaje del perceptrón

El algoritmo de Rosenblatt recorre los ejemplos y corrige solo cuando se equivoca:

```
para cada época:
    para cada ejemplo (x, y):
        ŷ = escalón(w · x + b)
        si ŷ ≠ y:
            w ← w + η · (y − ŷ) · x
            b ← b + η · (y − ŷ)
```

Con η (tasa de aprendizaje) > 0, la corrección empuja el hiperplano hacia el ejemplo mal clasificado: si y = 1 y ŷ = 0, suma x a los pesos (acerca la frontera); si y = 0 y ŷ = 1, resta x. El **teorema de convergencia del perceptrón** (Novikoff, 1962) garantiza que si los datos son linealmente separables con margen γ > 0, el algoritmo converge en un número finito de actualizaciones, acotado por $(R/\gamma)^2$, donde R es el radio de los datos.

🚧 Separabilidad lineal y el problema XOR

Un conjunto es **linealmente separable** si existe un hiperplano que deja todas las instancias positivas a un lado y las negativas al otro. XOR no lo es. Prueba por contradicción con las cuatro entradas booleanas (y = 1 para (0,1) y (1,0)):

```
(0,0) → 0:  b < 0
(0,1) → 1:  w₂ + b ≥ 0
(1,0) → 1:  w₁ + b ≥ 0
(1,1) → 0:  w₁ + w₂ + b < 0

Sumando las filas 2 y 3:  w₁ + w₂ + 2b ≥ 0  →  w₁ + w₂ + b ≥ −b > 0
Contradicción con la fila 4. No existe (w₁, w₂, b) que satisfaga las cuatro.
```

Minsky y Papert (*Perceptrons*, 1969) formalizaron esta y otras limitaciones (paridad, conectividad), lo que redujo drásticamente la financiación del conexionismo hasta que el MLP con backpropagation (clase 050) mostró que **componer** unidades no lineales en capas resuelve XOR y mucho más.

🧮 Ejemplo trabajado

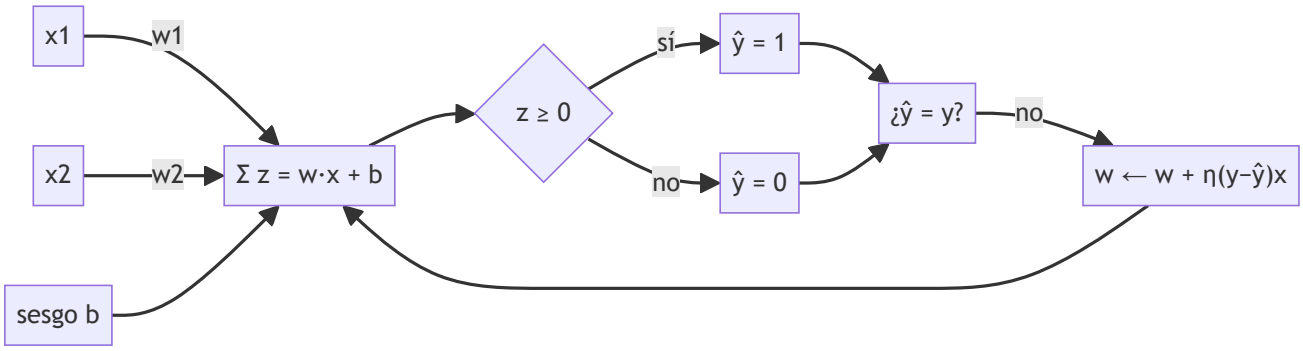
Entrenamos un perceptrón para la función AND con w = (0, 0), b = 0, η = 1 y regla escalón(z ≥ 0 → 1). Primeras dos épocas (solo se muestran los pasos con actualización):

Paso	x	y	z = w·x+b	ŷ	error (y−ŷ)	w nuevo	b nuevo
1.1	(0,0)	0	0	1	−1	(0,0)	−1
1.4	(1,1)	1	−1	0	+1	(1,1)	0
2.1	(0,0)	0	0	1	−1	(1,1)	−1
2.2	(0,1)	0	0	1	−1	(1,0)	−2
2.4	(1,1)	1	−1	0	+1	(2,1)	−1

Continuando el mismo procedimiento, el algoritmo converge en la época 6 con **w = (2, 1), b = −3**: comprueba que $2x_1 + x_2 - 3 \geq 0$ solo para (1,1). La frontera de decisión es la recta $2x_1 + x_2 = 3$. Si repites el proceso con XOR, las actualizaciones ciclan indefinidamente: no hay hiperplano que encontrar.

📊 Propiedades y comparación

Modelo	Frontera	Salida	Entrenamiento	Converge si...
Perceptrón	lineal	binaria dura (0/1)	regla de error, online	datos separables (garantía finita)
Regresión logística	lineal	probabilidad σ(z)	descenso de gradiente sobre pérdida convexa	siempre (mínimo global)
SVM lineal	lineal de margen máximo	binaria + margen	optimización convexa	siempre
MLP (clase 050)	no lineal	flexible	backpropagation, no convexo	sin garantía global



⚠️ Errores conceptuales frecuentes

1. **"El perceptrón minimiza una función de pérdida diferenciable."** No: la regla de Rosenblatt es una corrección por error con salida escalón, no descenso de gradiente sobre una pérdida suave (eso es la regresión logística o ADALINE).
2. **"Si el perceptrón no converge, hay que entrenar más épocas."** Si los datos no son linealmente separables, ninguna cantidad de épocas ayuda: el algoritmo cicla.
3. **"XOR demostró que las redes neuronales no sirven."** Demostró que *una capa* no basta; un MLP con una capa oculta de 2 neuronas resuelve XOR exactamente.

4. **"El sesgo b es opcional."** Sin sesgo, la frontera pasa por el origen y problemas triviales (p. ej. AND) pueden volverse irresolubles según la codificación.
5. **"Más features siempre ayudan a separar."** Proyectar a dimensiones altas puede lograr separabilidad, pero también memorización sin generalización: separar el conjunto de entrenamiento no implica clasificar bien datos nuevos.

🚀 Del aprendizaje a la operación

Entre este núcleo educativo y un clasificador real faltan: features numéricas estandarizadas (no entradas booleanas de juguete), validación con datos separados del entrenamiento, una pérdida calibrada en probabilidad si la decisión tiene costos asimétricos, y monitoreo de deriva de datos. Hoy nadie despliega un perceptrón puro, pero su regla de actualización sobrevive en cada capa lineal de una red moderna.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("neural")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Rosenblatt, F. (1958). *The perceptron: a probabilistic model for information storage and organization in the brain*. Psychological Review, 65(6). doi:10.1037/h0042519 — uso: fuente primaria del mecanismo estudiado
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*, cap. 6 (Deep Feedforward Networks). deeplearningbook.org/contents/mlp.html — uso: desarrollo extendido del tema
- Bishop, C. (2006). *Pattern Recognition and Machine Learning*, cap. 4 (Linear Models for Classification). PDF oficial gratuito — uso: referencia consultada en su fuente original
- Documentación de PyTorch: `torch.nn.Linear` — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P01 · El perceptrón: un modelo probabilístico de almacenamiento y organización de información en el cerebro	1958	Primera máquina que aprende sus propios pesos a partir de ejemplos en lugar de ejecutar reglas escritas por una persona.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 6 · obra de referencia de la parte 04
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 04 · modelos profundos desde la probabilidad

📝 Evaluación completa

❓ Preguntas

1. Define perceptrón y límites de separabilidad sin usar una marca o framework como definición.
2. Explica la relación entre perceptrón, separabilidad, pesos, sesgo.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

050 — MLP y backpropagation

Parte: 04 — Redes neuronales y deep learning
Nivel: intermedio-avanzado · **Horas estimadas:** 6
Laboratorio: `neural` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **mlp y backpropagation** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar mlp y backpropagation usando los conceptos `MLP`, `backpropagation`, `gradiente`, `pérdida`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`MLP`, `backpropagation`, `gradiente`, `pérdida`

🗺️ Ubicación en el mapa de la IA

El perceptrón multicapa (MLP) rompe el límite de separabilidad lineal de la clase 049: componer capas de unidades no lineales permite aproximar funciones arbitrarias. Backpropagation (Rumelhart, Hinton y Williams, 1986) hizo entrenable esa composición al calcular gradientes de forma eficiente con la regla de la cadena, y es el motor de todo lo que sigue en esta parte: CNN, RNN, Transformers y modelos generativos se entrenan exactamente con este algoritmo.

📖 Fundamentos

🪜 El perceptrón multicapa

Un MLP alterna transformaciones afines y no linealidades:

```
h⁽¹⁾ = f(W⁽¹⁾ x + b⁽¹⁾)          capa oculta
ŷ    = g(W⁽²⁾ h⁽¹⁾ + b⁽²⁾)       capa de salida
```

La no linealidad f (sigmoide, tanh, ReLU) es imprescindible: sin ella, la composición de capas lineales colapsa en una única transformación lineal. El **teorema de aproximación universal** (Cybenko, 1989; Hornik, 1991) garantiza que un MLP con una capa oculta suficientemente ancha aproxima cualquier función continua sobre un compacto con precisión arbitraria — pero no dice cuántas neuronas hacen falta ni cómo encontrar los pesos: eso lo resuelve (parcialmente) el entrenamiento por gradiente.

🎯 Función de pérdida y descenso de gradiente

El entrenamiento minimiza una pérdida $L(\theta)$ sobre los datos. Para regresión, el error cuadrático $L = \tfrac{1}{2}(\hat{y} - t)^2$; para clasificación, la entropía cruzada. El descenso de gradiente actualiza cada parámetro en contra de su derivada parcial:

```
θ ← θ − η · ∂L/∂θ
```

El problema es calcular $\partial L/\partial \theta$ para millones de parámetros distribuidos en capas.

🔗 Backpropagation: la regla de la cadena organizada

Backpropagation calcula todos los gradientes en dos pasadas sobre el **grafo de cómputo**:

```
1. Forward: evaluar la red guardando los valores intermedios (z, h, ŷ).
2. Backward: propagar δ = ∂L/∂z de la salida hacia la entrada:
   δ_salida = ∂L/∂ŷ · g'(z_salida)
   δ_capa   = (W^T_siguiente · δ_siguiente) ⊙ f'(z_capa)
   ∂L/∂W    = δ · h^T_anterior          ∂L/∂b = δ
```

Cada gradiente local se calcula una sola vez y se reutiliza (programación dinámica): el coste del backward es del mismo orden que el forward, O(número de pesos). Esto es lo que hace viable entrenar redes profundas; los frameworks modernos (autograd de PyTorch) construyen el grafo y aplican estas fórmulas automáticamente.

🕳️ Por qué el paisaje no es convexo

La pérdida de un MLP tiene simetrías (permutar neuronas ocultas no cambia la función) y no linealidades que crean múltiples mínimos y puntos de silla. No hay garantía de mínimo global; en la práctica, el descenso estocástico con buenas inicializaciones (clase 051) y optimizadores (clase 052) encuentra soluciones útiles.

🧮 Ejemplo trabajado

Red 2-2-1: entrada $x = (1, 0.5)$, oculta sigmoide σ, salida lineal, pérdida $L = \tfrac{1}{2}(\hat{y}-t)^2$ con objetivo $t = 1$. Pesos: $W^{(1)} = [[0.1, 0.2], [0.3, 0.4]]$, $b^{(1)} = (0,0)$, $w^{(2)} = (0.5, -0.5)$, $b^{(2)} = 0$.

Forward:

```
z₁ = 0.1·1 + 0.2·0.5 = 0.20  →  h₁ = σ(0.20) = 0.5498
z₂ = 0.3·1 + 0.4·0.5 = 0.50  →  h₂ = σ(0.50) = 0.6225
ŷ  = 0.5·0.5498 − 0.5·0.6225 = −0.0363
L  = ½(−0.0363 − 1)² = 0.5370
```

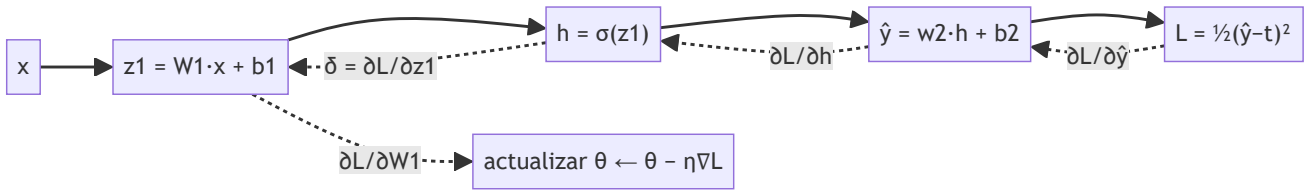
Backward (usando $\sigma'(z) = h(1-h)$):

```
∂L/∂ŷ  = ŷ − t = −1.0363
∂L/∂w⁽²⁾ = (ŷ−t)·h = (−0.5698, −0.6451)      ∂L/∂b⁽²⁾ = −1.0363
∂L/∂h₁ = (ŷ−t)·0.5  = −0.5182                ∂L/∂h₂ = (ŷ−t)·(−0.5) = +0.5182
σ'(z₁) = 0.5498·0.4502 = 0.2475              σ'(z₂) = 0.6225·0.3775 = 0.2350
δ₁ = −0.5182·0.2475 = −0.1283                δ₂ = +0.5182·0.2350 = +0.1218
∂L/∂W⁽¹⁾ = [δ₁·x ; δ₂·x] = [[−0.1283, −0.0641], [0.1218, 0.0609]]
```

Actualización con η = 0.1: $w^{(2)} \leftarrow (0.5570, -0.4355)$, $b^{(2)} \leftarrow 0.1036$, etc. Un segundo forward con estos pesos da una pérdida menor: el gradiente funcionó.

📊 Propiedades y comparación

Aspecto	Perceptrón (049)	MLP + backprop	Diferenciación numérica
Frontera	lineal	no lineal arbitraria	—
Coste del gradiente	no aplica	O(pesos), 1 backward	O(pesos²), 1 forward por peso
Exactitud del gradiente	—	exacta (analítica)	aproximada (error de truncamiento)
Garantía de convergencia	sí, si separable	no (paisaje no convexo)	no
XOR	imposible	2 neuronas ocultas bastan	—



⚠️ Errores conceptuales frecuentes

1. **"Backpropagation es un algoritmo de aprendizaje."** Es solo el cálculo eficiente del gradiente; el aprendizaje lo hace el optimizador (SGD, Adam) que usa ese gradiente.
2. **"Más capas siempre aproximan mejor."** El teorema universal ya se cumple con una capa; la profundidad ayuda a la *eficiencia* de la representación, pero agrava los problemas de gradiente (clases 051 y 054).
3. **"El gradiente indica la dirección al mínimo global."** Indica el descenso más rápido *local*; en paisajes no convexos puede llevar a mínimos locales o sillas.
4. **"Sin activaciones no lineales, una red profunda sigue siendo profunda."** $W_3(W_2(W_1x)) = (W_3W_2W_1)x$: colapsa a una sola capa lineal.
5. **"Backprop necesita derivar la red a mano."** La diferenciación automática en modo reverso aplica la regla de la cadena sobre el grafo de cómputo sin derivación manual.

🚀 Del aprendizaje a la operación

Entre este backward a mano y entrenar un modelo real faltan: minibatches y vectorización en GPU, inicialización y normalización correctas (clase 051), un optimizador con momento (clase 052), regularización

contra el sobreajuste, y validación honesta con datos nunca vistos. La matemática, sin embargo, es exactamente la de esta clase: cada `loss.backward()` de PyTorch ejecuta estas fórmulas.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("neural")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Rumelhart, D., Hinton, G. y Williams, R. (1986). *Learning representations by back-propagating errors*. Nature, 323. doi:10.1038/323533a0 — uso: fuente primaria del mecanismo estudiado
- Cybenko, G. (1989). *Approximation by superpositions of a sigmoidal function*. doi:10.1007/BF02551274 — uso: fuente primaria del mecanismo estudiado
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*, cap. 6 (Deep Feedforward Networks). deeplearningbook.org/contents/mlp.html — uso: desarrollo extendido del tema
- Documentación de PyTorch: mecánica de autograd — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P02 · Aprender representaciones retropropagando errores	1986	Un procedimiento práctico para entrenar capas ocultas: la red descubre representaciones intermedias que nadie diseñó.	notebook
P41 · Adam: un método de optimización estocástica	2014	Un paso de aprendizaje por dimensión, adaptado a la escala de su propio gradiente. Es el optimizador por defecto de casi todo lo que vino después.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 6 · obra de referencia de la parte 04
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 04 · modelos profundos desde la probabilidad

📝 Evaluación completa

❓ Preguntas

1. Define mlp y backpropagation sin usar una marca o framework como definición.
2. Explica la relación entre MLP, backpropagation, gradiente, pérdida.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;

- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

051 — Activaciones, inicialización y normalización

Parte: 04 — Redes neuronales y deep learning
Nivel: intermedio-avanzado · **Horas estimadas:** 6
Laboratorio: `neural` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **activaciones, inicialización y normalización** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar activaciones, inicialización y normalización usando los conceptos `ReLU`, `GELU`, `inicialización`, `normalización`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`ReLU`, `GELU`, `inicialización`, `normalización`

🗺️ Ubicación en el mapa de la IA

Backpropagation (clase 050) funciona en teoría con cualquier activación y pesos iniciales, pero en la práctica las redes profundas no entrenaban hasta que se entendió cómo mantener sanas las señales y los gradientes capa a capa. ReLU, la inicialización de Xavier/He y batch normalization son las tres piezas que, entre 2010 y 2015, convirtieron "redes de 3 capas" en "redes de 100+ capas" y habilitaron CNN profundas (clase 053) y Transformers (clase 055).

📖 Fundamentos

⚡ Funciones de activación

Función	Fórmula	Rango	Derivada	Problema típico
Sigmoide	$\sigma(z) = 1/(1+e^{-z})$	$(0,1)$	$\sigma(1-\sigma) \leq 0.25$	saturación: gradiente ≈ 0 en las colas
Tanh	$\tanh(z)$	$(-1,1)$	$1-\tanh^2 \leq 1$	satura, pero centrada en 0
ReLU	$\max(0, z)$	$[0,\infty)$	0 o 1	"neuronas muertas" ($z<0$ siempre)
Leaky ReLU	$\max(\alpha z, z)$, $\alpha \approx 0.01$	$(-\infty,\infty)$	α o 1	mitiga neuronas muertas
GELU	$z \cdot \Phi(z)$	$(-0.17,\infty)$ aprox.	suave	coste ligeramente mayor; estándar en Transformers

La sigmoide multiplica el gradiente por ≤ 0.25 en cada capa: tras 10 capas, el gradiente puede reducirse en $0.25^{10} \approx 10^{-6}$ (gradiente que desaparece). ReLU tiene derivada 1 en la zona activa, lo que preserva la magnitud del gradiente y además produce activaciones dispersas (muchos ceros exactos).

🎲 Inicialización: Xavier/Glorot y He

Si los pesos iniciales son demasiado grandes, las activaciones explotan o saturan; demasiado pequeños, la señal se extingue. El criterio es conservar la **varianza** de la señal al atravesar cada capa. Para $z = \Sigma\, w_i x_i$ con n entradas independientes:

```
Var(z) = n · Var(w) · Var(x)
```

Para que Var(z) ≈ Var(x) hace falta Var(w) = 1/n. Equilibrando forward y backward:

```
Xavier/Glorot (tanh, sigmoide):  Var(w) = 2 / (n_in + n_out)
He (ReLU):                        Var(w) = 2 / n_in
```

El factor 2 de He compensa que ReLU anula la mitad de las activaciones (mitad de la varianza). Con estas fórmulas, una red de decenas de capas mantiene señales de magnitud estable desde el primer paso de entrenamiento.

📏 Normalización: batch norm y layer norm

Batch normalization (Ioffe y Szegedy, 2015) estandariza cada activación usando las estadísticas del minibatch y reintroduce escala y desplazamiento aprendibles:

```
μ = media del batch      σ² = varianza del batch
x̂ᵢ = (xᵢ − μ) / √(σ² + ε)
yᵢ = γ · x̂ᵢ + β          (γ, β aprendibles)
```

Efectos: permite tasas de aprendizaje mayores, reduce la sensibilidad a la inicialización y actúa como regularizador suave (el ruido de las estadísticas del batch). En inferencia se usan medias móviles acumuladas, no las del batch — por eso `model.eval()` cambia el comportamiento. **Layer normalization** (Ba et al., 2016) normaliza sobre las features de *cada ejemplo* en vez de sobre el batch: no depende del tamaño de batch y es la opción estándar en RNN y Transformers.

🧮 Ejemplo trabajado

Inicialización. Capa de 256 → 256 unidades:

```
Xavier: Var(w) = 2/(256+256) = 1/256   →  std = 1/16 = 0.0625
He:     Var(w) = 2/256                 →  std = 0.0884
```

Si en cambio usaras std = 1: Var(z) = 256 → std(z) = 16; con sigmoide, prácticamente todas las unidades saturan y el gradiente se anula desde el paso 1.

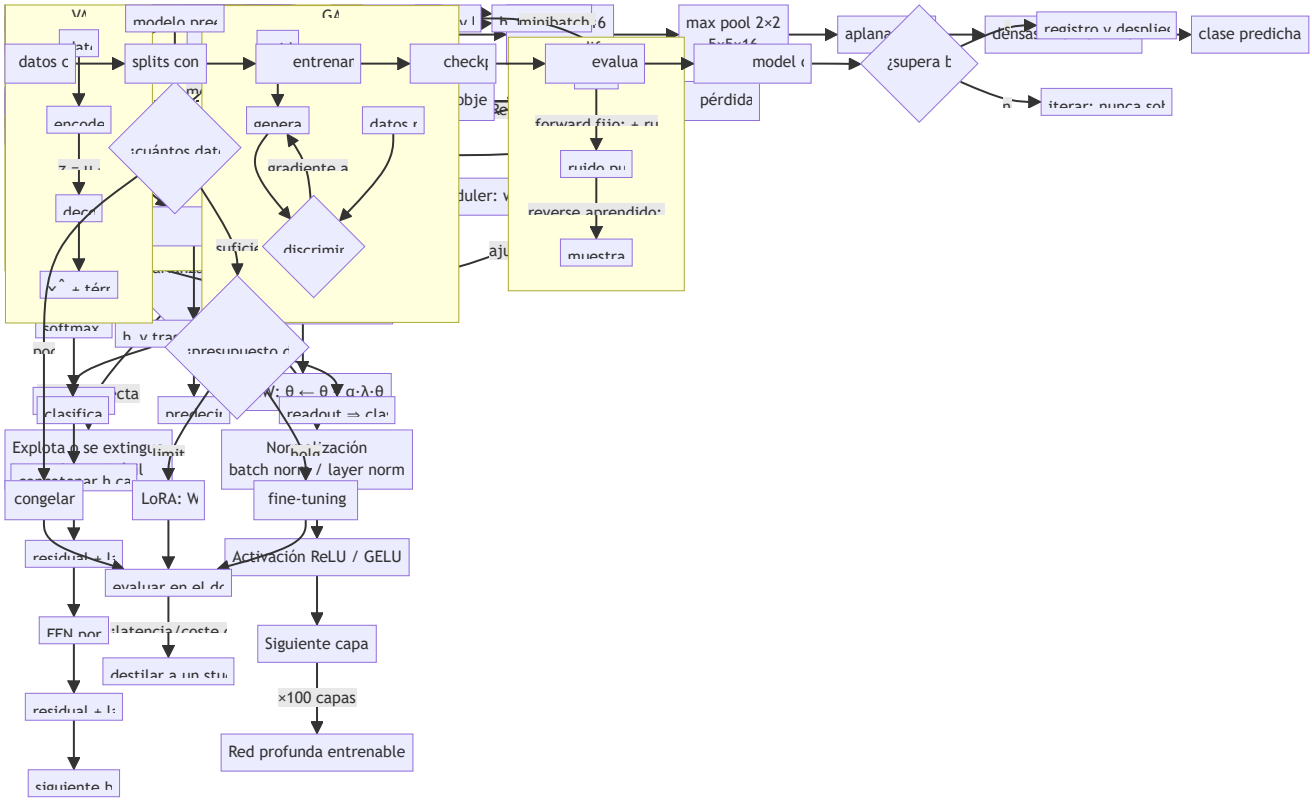
Batch norm a mano. Batch de 4 activaciones $x = (1, 2, 3, 4)$, $\gamma = 2$, $\beta = 1$, $\varepsilon \approx 0$:

```
μ = 2.5      σ² = ((-1.5)² + (-0.5)² + 0.5² + 1.5²)/4 = 1.25      √σ² = 1.1180
x̂ = (-1.3416, -0.4472, +0.4472, +1.3416)
y  = 2·x̂ + 1 = (-1.6833, +0.1056, +1.8944, +3.6833)
```

La salida tiene media $\beta = 1$ y desviación $\gamma = 2$ exactas: la red controla la distribución de sus activaciones con dos parámetros aprendidos.

📊 Propiedades y comparación

Técnica	Qué estabiliza	Cuándo se aplica	Coste	Limitación
ReLU/GELU	gradiente (derivada ≈ 1)	siempre (diseño)	nulo	neuronas muertas (ReLU)
Xavier/He	varianza inicial de señales	solo en t=0	nulo	el entrenamiento puede degradarla
Batch norm	distribución de activaciones	cada forward	~30 % más lento	depende del tamaño de batch
Layer norm	ídem, por ejemplo	cada forward	similar	menos eficaz en CNN pequeñas



⚠️ Errores conceptuales frecuentes

1. **"La inicialización da igual porque el entrenamiento la corrige."** Con una mala escala inicial el gradiente desaparece o explota en el primer paso: no hay señal con la que corregir nada.
2. **"ReLU no es diferenciable en 0, así que backprop falla."** El punto único de no diferenciabilidad tiene medida cero; se toma subgradiente 0 o 1 y funciona.
3. **"Batch norm normaliza los datos de entrada."** Normaliza activaciones *internas* de cada capa, con estadísticas del minibatch; la estandarización de entrada es otra cosa (preprocesamiento).
4. **"Batch norm se comporta igual en entrenamiento e inferencia."** No: en inferencia usa medias móviles acumuladas. Olvidar `model.eval()` es un bug clásico.
5. **"Inicializar todo a cero es lo más neutro."** Con pesos idénticos todas las neuronas de una capa reciben el mismo gradiente y aprenden lo mismo para siempre (falta de ruptura de simetría).

🚀 Del aprendizaje a la operación

En un sistema real estas decisiones vienen empaquetadas: `torch.nn.init` aplica Xavier/He según la activación declarada, y las arquitecturas estándar ya traen la normalización correcta para su dominio. Lo que sigue siendo trabajo del ingeniero es diagnosticar cuándo fallan: activaciones saturadas, pérdida que no baja en el paso 1, o métricas que cambian entre `train()` y `eval()` apuntan exactamente a los conceptos de esta clase.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("neural")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Glorot, X. y Bengio, Y. (2010). *Understanding the difficulty of training deep feedforward neural networks*. AISTATS. proceedings.mlr.press/v9/glorot10a.html — uso: referencia consultada en su fuente original
- He, K. et al. (2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. arXiv:1502.01852 — uso: fuente primaria del mecanismo estudiado
- Ioffe, S. y Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. arXiv:1502.03167 — uso: fuente primaria del mecanismo estudiado
- Ba, J., Kiros, J. y Hinton, G. (2016). *Layer Normalization*. arXiv:1607.06450 — uso: fuente primaria del mecanismo estudiado
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*, cap. 8. deeplearningbook.org/contents/optimization.html — uso: desarrollo extendido del tema
- Documentación de PyTorch: `torch.nn.init` — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P02 · Aprender representaciones retropropagando errores	1986	Un procedimiento práctico para entrenar capas ocultas: la red descubre representaciones intermedias que nadie diseñó.	notebook
P40 · Dropout: una forma simple de evitar el sobreajuste en redes neuronales	2014	Apagar unidades al azar durante el entrenamiento equivale a entrenar un ensamblado exponencial de subredes que comparten pesos.	notebook
P43 · Normalización por lotes: acelerar el entrenamiento profundo	2015	Normalizar las activaciones dentro de la red permite tasas de aprendizaje mucho mayores y hace el entrenamiento profundo mucho menos frágil.	notebook
P44 · Aprendizaje residual profundo para reconocimiento de imágenes	2015	El atajo identidad hace apilables cientos de capas. Es la misma idea aditiva de la LSTM, aplicada a la profundidad.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 8 · obra de referencia de la parte 04
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 04 · modelos profundos desde la probabilidad

📝 Evaluación completa

❓ Preguntas

1. Define activaciones, inicialización y normalización sin usar una marca o framework como definición.
2. Explica la relación entre ReLU, GELU, inicialización, normalización.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.

- [ ] Se declara al menos un riesgo o condición de no uso.

---

052 — Optimizadores, regularización y schedulers

Parte: 04 — Redes neuronales y deep learning
Nivel: intermedio-avanzado · **Horas estimadas:** 6
Laboratorio: `optimization` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **optimizadores, regularización y schedulers** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar optimizadores, regularización y schedulers usando los conceptos `SGD`, `AdamW`, `dropout`, `scheduler`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`SGD`, `AdamW`, `dropout`, `scheduler`

🗺️ Ubicación en el mapa de la IA

Con el gradiente ya disponible (clase 050) y señales estables (clase 051), queda la pregunta operativa central del deep learning: *cómo usar ese gradiente*. SGD con momento, Adam/AdamW, dropout, weight decay y los schedulers de tasa de aprendizaje son el "manual de vuelo" con el que se entrenan desde CNN hasta los LLM actuales; casi cualquier receta de entrenamiento moderna es una combinación de estas piezas.

📖 Fundamentos

📉 SGD y minibatches

El descenso de gradiente **estocástico** estima el gradiente con un minibatch de B ejemplos en lugar del dataset completo:

```
θ ← θ − η · (1/B) Σ_{i∈batch} ∇L_i(θ)
```

El ruido del muestreo abarata cada paso y, además, ayuda a escapar de puntos de silla. B pequeño = más ruido y más pasos; B grande = gradiente más fiel pero pasos más caros y a veces peor generalización.

🎳 Momento (momentum)

El momento acumula una media móvil de gradientes, como una bola con inercia:

```
v ← μ·v + ∇L(θ)        (μ ≈ 0.9)
θ ← θ − η·v
```

Acelera en direcciones consistentes (el valle) y amortigua oscilaciones en direcciones que cambian de signo (las paredes del valle). Nesterov evalúa el gradiente en la posición "anticipada" $\theta - \eta\mu v$, con corrección más fina.

🧭 Adam y AdamW

Adam (Kingma y Ba, 2014) mantiene medias móviles del gradiente (m, primer momento) y de su cuadrado (v, segundo momento), y adapta la escala por parámetro:

```
m ← β₁·m + (1−β₁)·g            (β₁ = 0.9)
v ← β₂·v + (1−β₂)·g²           (β₂ = 0.999)
m̂ = m/(1−β₁ᵗ)    v̂ = v/(1−β₂ᵗ)     ← corrección de sesgo inicial
θ ← θ − α · m̂ / (√v̂ + ε)
```

La división por $\sqrt{\hat{v}}$ hace que parámetros con gradientes históricamente grandes den pasos pequeños y viceversa: cada parámetro tiene su tasa efectiva. **AdamW** (Loshchilov y Hutter, 2017) separa el weight decay de la actualización adaptativa (`θ ← θ − α·λ·θ` aparte), porque mezclar L2 dentro de Adam lo distorsiona; AdamW es el estándar de facto para Transformers.

🛡️ Regularización

- **Weight decay / L2**: penaliza $\|\theta\|^2$ empujando los pesos hacia cero; controla la complejidad efectiva del modelo.
- **Dropout** (Srivastava et al., 2014): durante el entrenamiento anula cada activación con probabilidad p y escala el resto por $1/(1-p)$ (*inverted dropout*); en inferencia no hace nada. Obliga a redundancia: ninguna neurona puede depender de otra concreta. Equivale a entrenar un ensamble implícito de subredes.
- **Early stopping**: detener cuando la métrica de *validación* deja de mejorar.
- **Aumento de datos**: regulariza atacando el problema en la fuente.

⏱️ Schedulers y warmup

La tasa de aprendizaje óptima cambia durante el entrenamiento. Recetas comunes: *step decay* (dividir η por 10 cada k épocas), *cosine annealing* (decaimiento suave hasta ~0) y **warmup** (crecer linealmente desde ~0 durante los primeros pasos, crítico en Transformers: al inicio las estimaciones $\hat{m}$, $\hat{v}$ de Adam son ruidosas y un η grande puede desestabilizar la red).

🧮 Ejemplo trabajado

Una actualización de Adam a mano (primer paso, t = 1): parámetro $\theta = 1.0$, gradiente $g = 0.5$, $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, $m_0 = v_0 = 0$.

```
m = 0.9·0 + 0.1·0.5   = 0.05
v = 0.999·0 + 0.001·0.25 = 0.00025
m̂ = 0.05 / (1-0.9¹)   = 0.05/0.1   = 0.5
v̂ = 0.00025 / (1-0.999¹) = 0.00025/0.001 = 0.25
Δθ = -0.001 · 0.5 / (√0.25 + 10⁻⁸) = -0.001 · 0.5/0.5 = -0.001
θ ← 1.0 - 0.001 = 0.999
```

Nota el efecto de la corrección de sesgo: sin ella, $m = 0.05$ y $v = 0.00025$ darían un paso distorsionado; con ella, el primer paso vale exactamente $-\alpha \cdot g/|g| = -\alpha$, es decir, Adam da pasos de tamaño $\approx \alpha$ independientes de la escala del gradiente.

Dropout a mano: activaciones $h = (2, 4, 6)$ con $p = 0.5$ y máscara $(1, 0, 1)$: salida entrenamiento = $(2/0.5, 0, 6/0.5) = (4, 0, 12)$. En inferencia: $(2, 4, 6)$ sin cambios — la esperanza coincide gracias al escalado $1/(1-p)$.

📊 Propiedades y comparación

Optimizador	Estado extra	Tasa por parámetro	Sensible a escala de g	Uso típico
SGD	ninguno	no	sí	visión (con momento), máxima simplicidad
SGD + momento	v (1× params)	no	sí	CNN clásicas; buena generalización
Adam	m, v (2× params)	sí	no (normaliza)	por defecto en la mayoría de tareas
AdamW	m, v (2× params)	sí	no	Transformers, LLM (con warmup + cosine)

⚠️ Errores conceptuales frecuentes

1. **"Adam siempre es mejor que SGD."** Adam converge más rápido en pasos, pero en visión SGD+momento bien ajustado generaliza a veces mejor; "mejor" depende de tarea y presupuesto de ajuste.
2. **"Dropout se aplica también en inferencia."** No: en inferencia se desactiva; el escalado $1/(1-p)$ durante el entrenamiento mantiene la esperanza correcta.

3. **"Weight decay y L2 son siempre lo mismo."** Coinciden en SGD puro; en Adam difieren (la L2 pasa por la normalización adaptativa) — esa es la razón de AdamW.
4. **"Si la pérdida de entrenamiento baja, todo va bien."** El sobreajuste se detecta en validación; entrenar más allá del punto de quiebre empeora el modelo real.
5. **"El warmup es un truco opcional."** En Transformers grandes, omitirlo produce divergencia temprana reproducible: las primeras estimaciones de $\hat{v}$ son tan ruidosas que los pasos iniciales pueden ser enormes.

🚀 Del aprendizaje a la operación

Un entrenamiento real añade: búsqueda de hiperparámetros con presupuesto explícito, *gradient clipping* para picos, precisión mixta (float16) con escalado de pérdida, checkpoints reanudables y registro de curvas de entrenamiento/validación. La receta "AdamW + warmup + cosine + weight decay" es un punto de partida sólido, no un dogma: cada dominio la recalibra empíricamente.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("optimization")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Kingma, D. y Ba, J. (2014). *Adam: A Method for Stochastic Optimization*. arXiv:1412.6980 — uso: fuente primaria del mecanismo estudiado
- Loshchilov, I. y Hutter, F. (2017). *Decoupled Weight Decay Regularization* (AdamW). arXiv:1711.05101 — uso: fuente primaria del mecanismo estudiado
- Srivastava, N. et al. (2014). *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. JMLR 15. jmlr.org/papers/v15/srivastava14a.html — uso: referencia consultada en su fuente original
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*, cap. 8 (Optimization). deeplearningbook.org/contents/optimization.html — uso: desarrollo extendido del tema
- Documentación de PyTorch: `torch.optim` — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P40 · Dropout: una forma simple de evitar el sobreajuste en redes neuronales	2014	Apagar unidades al azar durante el entrenamiento equivale a entrenar un ensamblado exponencial de subredes que comparten pesos.	notebook
P41 · Adam: un método de optimización estocástica	2014	Un paso de aprendizaje por dimensión, adaptado a la escala de su propio gradiente. Es el optimizador por defecto de casi todo lo que vino después.	notebook
P43 · Normalización por lotes: acelerar el entrenamiento profundo	2015	Normalizar las activaciones dentro de la red permite tasas de aprendizaje mucho mayores y hace el entrenamiento profundo mucho menos frágil.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 8 · obra de referencia de la parte 04
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 04 · modelos profundos desde la probabilidad

📝 Evaluación completa

❓ Preguntas

1. Define optimizadores, regularización y schedulers sin usar una marca o framework como definición.
2. Explica la relación entre SGD, AdamW, dropout, scheduler.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

053 — CNN y aprendizaje espacial

Parte: 04 — Redes neuronales y deep learning
Nivel: intermedio-avanzado · **Horas estimadas:** 6
Laboratorio: `perception` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **cnn y aprendizaje espacial** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cnn y aprendizaje espacial usando los conceptos `convolución`, `pooling`, `receptive field`, `visión`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`convolución`, `pooling`, `receptive field`, `visión`

🗺️ Ubicación en el mapa de la IA

Las redes convolucionales incorporan al MLP (clase 050) un sesgo inductivo espacial: los patrones visuales son locales e invariantes a traslación. De LeNet-5 (LeCun, 1998, lectura de cheques) a AlexNet (2012, punto de inflexión del deep learning moderno) y ResNet (He et al., 2015, redes de 100+ capas), las CNN demostraron que aprender las features supera a diseñarlas a mano, y su vocabulario (stride, padding, campos receptivos) reaparece en los Transformers de visión.

📖 Fundamentos

🔲 La operación de convolución

Una capa convolucional desliza un **kernel** (filtro) de pesos K sobre la entrada I y calcula en cada posición un producto punto local (en deep learning se implementa como correlación cruzada, sin voltear el kernel):

```
S(i, j) = Σₘ Σₙ I(i+m, j+n) · K(m, n) + b
```

Frente a una capa densa, la convolución impone dos restricciones que son su fuerza:

- **Conectividad local**: cada salida mira solo una ventana K×K de la entrada.
- **Pesos compartidos**: el mismo kernel se aplica en todas las posiciones → detecta el mismo patrón esté donde esté (equivarianza a traslación) y reduce drásticamente los parámetros.

El **tamaño de salida** con entrada N×N, kernel K×K, padding P y stride S:

```
salida = ⌊(N − K + 2P) / S⌋ + 1
```

Una capa tiene C_out filtros, cada uno de tamaño K×K×C_in: parámetros = C_out·(K·K·C_in + 1).

🌊 Pooling y jerarquía de features

Max pooling (o promedio) reduce la resolución tomando el máximo de cada ventana (típicamente 2×2, stride 2). Aporta invarianza local a pequeñas traslaciones y reduce cómputo. Apilando conv + pooling, el **campo receptivo** (la región de la imagen original que influye en una activación) crece capa a capa: las primeras capas detectan bordes y texturas; las intermedias, partes; las profundas, objetos. Para capas convolucionales apiladas, el campo receptivo crece según RF_l = RF_{l−1} + (K−1)·Πproducto de strides anteriores; dos capas 3×3 "ven" 5×5, tres capas 3×3 "ven" 7×7 — con menos parámetros y más no linealidad que un único 7×7 (la razón del diseño de VGG).

🏛️ De LeNet a ResNet

```
LeNet-5 (1998):   conv 5×5 + pooling ×2 → densas. ~60k parámetros, dígitos MNIST.
AlexNet (2012):   ReLU + dropout + GPU. Ganó ImageNet por >10 puntos de error.
VGG (2014):       solo kernels 3×3 apilados; profundidad regular (16-19 capas).
ResNet (2015):    bloques residuales y = F(x) + x. 152 capas entrenables.
```

El **bloque residual** de ResNet es el aporte conceptual clave: en lugar de aprender la transformación completa H(x), la capa aprende el residuo F(x) = H(x) − x. El atajo (skip connection) da al gradiente una autopista directa hacia las capas tempranas y resuelve la *degradación* (redes más profundas entrenaban peor que las someras incluso en entrenamiento, sin ser sobreajuste). Las skip connections son hoy ubicuas: también los Transformers las usan en cada bloque.

🧮 Ejemplo trabajado

Convolución 2×2 a mano (stride 1, sin padding). Entrada 4×4 y kernel:

```
I = | 1 2 0 1 |        K = |  1  0 |
    | 0 1 3 2 |            |  0 -1 |
    | 1 0 2 1 |
    | 2 1 0 1 |
```

Cada salida es I(i,j)·1 + I(i+1,j+1)·(−1) (los otros dos pesos son 0). Salida 3×3 (tamaño: (4−2+0)/1 + 1 = 3):

```
S = | 1-1  2-3  0-2 |   | 0 -1 -2 |
    | 0-0  1-2  3-1 | = | 0 -1  2 |
    | 1-1  0-0  2-1 |   | 0  0  1 |
```

Este kernel responde a diferencias diagonales: un detector de borde primitivo.

Tamaños y parámetros (estilo LeNet): entrada 32×32×3, capa de 6 filtros 5×5, stride 1, sin padding → salida (32−5)/1+1 = **28×28×6**; parámetros = 6·(5·5·3 + 1) = **456**. Una capa densa equivalente (3072 → 4704 unidades) necesitaría ~14.5 millones de pesos: la compartición de pesos reduce 5 órdenes de magnitud.

📊 Propiedades y comparación

Aspecto	Capa densa	Convolución	Pooling
Parámetros	n_in·n_out	C_out·(K²·C_in+1)	0
Estructura espacial	la destruye	la preserva	la reduce con invarianza
Invarianza a traslación	no	equivarianza	invarianza local
Campo receptivo	global	local, crece con profundidad	duplica el crecimiento
Uso típico	cabeza clasificadora	extracción de features	reducción de resolución

⚠️ Errores conceptuales frecuentes

1. **"La convolución de deep learning voltea el kernel."** Los frameworks implementan correlación cruzada; como los pesos se aprenden, la distinción es irrelevante en la práctica (el kernel aprendido absorbe el volteo).
2. **"Pooling aprende parámetros."** Max/average pooling no tiene pesos; solo reduce resolución. Hoy a menudo se sustituye por convoluciones con stride 2.
3. **"Más profundidad siempre mejoraba antes de ResNet."** Al contrario: la *degradación* (peor error de entrenamiento al profundizar) era el bloqueo; el bloque residual la resolvió, no el sobreajuste.
4. **"El campo receptivo de una neurona es su kernel."** Es la región de la *imagen original* que la influye, y crece con cada capa apilada: dos 3×3 ven 5×5.
5. **"Las CNN son invariantes a rotación y escala."** Solo tienen equivarianza a traslación; rotaciones y escalas se atacan con aumento de datos.

🚀 Del aprendizaje a la operación

Un sistema de visión real parte casi siempre de una CNN (o ViT) preentrenada en ImageNet y ajusta sobre el dominio propio (clase 059); añade aumento de datos, evaluación por clase (no solo accuracy global), calibración de confianza y pruebas frente a cambios de cámara, iluminación y distribución. La convolución de esta clase es la misma; lo que cambia es todo el andamiaje de datos y evaluación alrededor.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("perception")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- LeCun, Y., Bottou, L., Bengio, Y. y Haffner, P. (1998). *Gradient-based learning applied to document recognition*. Proc. IEEE. doi:10.1109/5.726791 — uso: fuente primaria del mecanismo estudiado
- Krizhevsky, A., Sutskever, I. y Hinton, G. (2017). *ImageNet classification with deep convolutional neural networks*. Comm. ACM. doi:10.1145/3065386 — uso: fuente primaria del mecanismo estudiado
- He, K., Zhang, X., Ren, S. y Sun, J. (2015). *Deep Residual Learning for Image Recognition*. arXiv:1512.03385 — uso: fuente primaria del mecanismo estudiado
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*, cap. 9 (Convolutional Networks). deeplearningbook.org/contents/convnets.html — uso: desarrollo extendido del tema
- Documentación de PyTorch: `torch.nn.Conv2d` — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P04 · Clasificación de ImageNet con redes neuronales convolucionales profundas	2012	El resultado que convirtió el deep learning en la corriente principal: margen amplio sobre los métodos de visión hechos a mano.	notebook
P42 · Explicar y aprovechar los ejemplos adversarios	2014	Una perturbación imperceptible cambia la predicción. Y la causa no es la profundidad: es la linealidad en dimensión alta.	notebook
P44 · Aprendizaje residual profundo para reconocimiento de imágenes	2015	El atajo identidad hace apilables cientos de capas. Es la misma idea aditiva de la LSTM, aplicada a la profundidad.	notebook
P46 · Una imagen vale 16x16 palabras: Transformers para reconocimiento de imágenes a escala	2020	Trata la imagen como una secuencia de parches y aplica un Transformer puro: la convolución deja de ser imprescindible en visión.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 9 · obra de referencia de la parte 04
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 04 · modelos profundos desde la probabilidad

📝 Evaluación completa

❓ Preguntas

1. Define cnn y aprendizaje espacial sin usar una marca o framework como definición.
2. Explica la relación entre convolución, pooling, receptive field, visión.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

054 — RNN, LSTM y secuencias

Parte: 04 — Redes neuronales y deep learning
Nivel: intermedio-avanzado · **Horas estimadas:** 6
Laboratorio: `neural` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **rnn, lstm y secuencias** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar rnn, lstm y secuencias usando los conceptos `RNN`, `LSTM`, `estado`, `secuencia`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`RNN`, `LSTM`, `estado`, `secuencia`

🗺️ Ubicación en el mapa de la IA

Las CNN explotan estructura espacial; las RNN explotan estructura *temporal*: un estado oculto que se actualiza paso a paso permite procesar secuencias de longitud variable (texto, audio, series). Su talón de Aquiles —los gradientes que desaparecen a lo largo del tiempo (Bengio et al., 1994)— motivó la LSTM (Hochreiter y Schmidhuber, 1997), dominante en NLP hasta 2017, cuando la atención (clase 055) eliminó la recurrencia. Entender por qué la LSTM funciona es entender por qué el Transformer ganó.

📖 Fundamentos

🔁 La red recurrente básica

Una RNN aplica la *misma* función en cada paso temporal, manteniendo un estado oculto h que resume el pasado:

```
h_t = tanh(W_x·x_t + W_h·h_{t-1} + b)
y_t = W_y·h_t + c
```

Los pesos (W_x, W_h) se comparten en el tiempo — el análogo temporal de la compartición espacial de las CNN. Desplegada (*unrolled*), una RNN de T pasos es una red profunda de T capas con pesos repetidos, y se entrena con **backpropagation through time (BPTT)**: se despliega el grafo y se aplica backprop normal, sumando los gradientes de cada paso sobre los mismos pesos compartidos.

📉 Gradientes que desaparecen (y explotan)

En BPTT, el gradiente que conecta el paso t con el paso t−k atraviesa k jacobianos:

```
∂h_t/∂h_{t-k} = Π_{i=1..k} diag(tanh'(z_{t-i+1})) · W_h
```

Si los valores singulares efectivos de ese producto son < 1, el gradiente se reduce geométricamente (**desaparece**: la red no aprende dependencias largas); si son > 1, crece geométricamente (**explota**: pasos de entrenamiento gigantes). La explosión se mitiga con *gradient clipping* (recortar la norma del gradiente); la desaparición exige cambiar la arquitectura.

🚪 LSTM: memoria con compuertas

La LSTM añade un **estado de celda** c_t con actualización *aditiva* y tres compuertas sigmoides (valores en (0,1)) que regulan el flujo de información:

```
f_t = σ(W_f·[h_{t-1}, x_t] + b_f)      compuerta de olvido
i_t = σ(W_i·[h_{t-1}, x_t] + b_i)      compuerta de entrada
c̃_t = tanh(W_c·[h_{t-1}, x_t] + b_c)   candidato
c_t = f_t ⊙ c_{t-1} + i_t ⊙ c̃_t         actualización aditiva de la celda
o_t = σ(W_o·[h_{t-1}, x_t] + b_o)      compuerta de salida
h_t = o_t ⊙ tanh(c_t)
```

La clave es c_t = f ⊙ c_{t−1} + i ⊙ c̃: el gradiente fluye por la celda a través de una suma modulada por f, no de una multiplicación repetida por W_h. Con f ≈ 1, la información (y el gradiente) puede conservarse durante cientos de pasos — el mismo principio del atajo residual de ResNet, aplicado al tiempo. La **GRU** (Cho et al., 2014) simplifica a dos compuertas fusionando celda y estado, con rendimiento similar y menos parámetros.

🚧 Limitación estructural

Aun con LSTM, la computación es inherentemente **secuencial** (h_t requiere h_{t−1}): no se paraleliza sobre la longitud de la secuencia, y toda la historia debe comprimirse en un vector de estado de tamaño fijo. Estas dos limitaciones son exactamente las que la auto-atención (clase 055) elimina.

🧮 Ejemplo trabajado

RNN escalar con w_h = 0.5, w_x = 1, b = 0, h₀ = 0 y entrada x = (1, 0, 0) — un impulso seguido de silencio:

```
h₁ = tanh(1·1 + 0.5·0)      = tanh(1.0)    = 0.7616
h₂ = tanh(1·0 + 0.5·0.7616) = tanh(0.3808) = 0.3634
h₃ = tanh(0.5·0.3634)       = tanh(0.1817) = 0.1797
```

La "memoria" del impulso decae geométricamente. El gradiente hacia atrás hace lo mismo: $|\partial h_3/\partial h_1| = |w_h \cdot \tanh'(z_3)| \cdot |w_h \cdot \tanh'(z_2)| \le 0.5^2 = 0.25$, y en general $\le w_h^k$ para k pasos: con w_h = 0.5, a 20 pasos

el gradiente es ≤ 10^{-6}.

Ahora una celda LSTM con f = 0.95, i = 0.5 constante y candidatos nulos tras el paso 1 ($c_1 = 1$): c_t = 0.95^(t−1) — tras 20 pasos conserva 0.377 (38 %), frente al 10^{-6} de la RNN: la actualización aditiva con compuerta de olvido cercana a 1 retiene señal órdenes de magnitud más tiempo.

📊 Propiedades y comparación

Aspecto	RNN simple	LSTM	GRU	Transformer (055)
Dependencias largas	pobres (gradiente ∝ w^k)	buenas (celda aditiva)	buenas	excelentes (acceso directo)
Parámetros por capa	1×	4×	3×	según d_model
Paralelización temporal	no	no	no	sí
Coste por paso	$O(d^2)$	$O(4d^2)$	$O(3d^2)$	O(n·d) por token
Memoria de contexto	vector fijo	vector fijo	vector fijo	todos los tokens

⚠️ Errores conceptuales frecuentes

1. **"La RNN tiene pesos distintos en cada paso temporal."** Comparte los mismos pesos en todos los pasos; por eso puede procesar longitudes arbitrarias.
2. **"El gradient clipping arregla los gradientes que desaparecen."** Solo mitiga los que *explotan*; la desaparición requiere arquitectura (LSTM/GRU) o atención.
3. **"La LSTM elimina el problema del gradiente por usar sigmoides."** Lo que lo mitiga es la actualización *aditiva* de la celda modulada por la compuerta de olvido, no el tipo de activación.
4. **"El estado oculto puede recordar toda la secuencia."** Es un vector de tamaño fijo: comprime con pérdida; secuencias largas y densas en información lo saturan.
5. **"Las LSTM están obsoletas y no vale la pena entenderlas."** Siguen siendo competitivas en series temporales pequeñas y streaming con baja latencia, y su mecánica de compuertas explica por qué la atención fue necesaria.

🚀 Del aprendizaje a la operación

Para producción con secuencias hoy se decide entre: LSTM/GRU (datos escasos, streaming, dispositivos limitados) o Transformers (corpus grandes, contexto largo). Faltarían además: truncated BPTT para secuencias largas, padding/masking correcto en batches, gradient clipping configurado y métricas específicas de secuencia (perplejidad, F1 por entidad), más pruebas con longitudes fuera del rango de entrenamiento.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("neural")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Hochreiter, S. y Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation 9(8). doi:10.1162/neco.1997.9.8.1735 — uso: fuente primaria del mecanismo estudiado
- Bengio, Y., Simard, P. y Frasconi, P. (1994). *Learning long-term dependencies with gradient descent is difficult*. IEEE Trans. Neural Networks. doi:10.1109/72.279181 — uso: fuente primaria del mecanismo estudiado
- Cho, K. et al. (2014). *Learning Phrase Representations using RNN Encoder-Decoder* (GRU). arXiv:1406.1078 — uso: fuente primaria del mecanismo estudiado
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*, cap. 10 (Sequence Modeling). deeplearningbook.org/contents/rnn.html — uso: desarrollo extendido del tema
- Documentación de PyTorch: `torch.nn.LSTM` — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P03 · Memoria larga de corto plazo	1997	Primera arquitectura recurrente capaz de mantener información a través de cientos de pasos sin que el gradiente se desvanezca.	notebook
P06 · Aprendizaje de secuencia a secuencia con redes neuronales	2014	Una única red aprende a mapear secuencias de longitud variable a secuencias de longitud variable, de extremo a extremo.	notebook
P07 · Traducción automática neuronal aprendiendo conjuntamente a alinear y traducir	2014	Nace la atención: el decodificador deja de depender de un único vector y consulta toda la entrada en cada paso.	notebook
P20 · Mamba: modelado de secuencias en tiempo lineal con espacios de estados selectivos	2023	El primer competidor serio del Transformer en lenguaje: tiempo lineal y estado de tamaño fijo, sin atención.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 10 · obra de referencia de la parte 04
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 04 · modelos profundos desde la probabilidad

📝 Evaluación completa

❓ Preguntas

1. Define rnn, lstm y secuencias sin usar una marca o framework como definición.
2. Explica la relación entre RNN, LSTM, estado, secuencia.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.

- [ ] Se declara al menos un riesgo o condición de no uso.

---

055 — Atención y arquitectura Transformer

Parte: 04 — Redes neuronales y deep learning
Nivel: intermedio-avanzado · **Horas estimadas:** 6
Laboratorio: `attention` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **atención y arquitectura transformer** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar atención y arquitectura transformer usando los conceptos `self-attention`, `multi-head`, `positional`, `transformer`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`self-attention`, `multi-head`, `positional`, `transformer`

🗺️ Ubicación en el mapa de la IA

"Attention Is All You Need" (Vaswani et al., 2017) eliminó la recurrencia de la clase 054: en lugar de comprimir la historia en un estado, cada token consulta directamente a todos los demás. El Transformer resolvió a la vez las dependencias largas y la paralelización, y es la arquitectura de BERT, GPT, los LLM actuales, la visión (ViT) y las proteínas (AlphaFold): probablemente la pieza de ingeniería más influyente de la IA moderna.

📖 Fundamentos

🔍 Atención escalada por producto punto

Cada token se proyecta en tres papeles: **query** Q (qué busco), **key** K (qué ofrezco como índice) y **value** V (qué contenido entrego):

```
Q = X·W_Q      K = X·W_K      V = X·W_V
Attention(Q, K, V) = softmax(Q·Kᵀ / √d_k) · V
```

Mecanismo paso a paso: (1) $Q \cdot K^T$ mide la afinidad de cada query con cada key (n×n puntuaciones); (2) se divide por $\sqrt{d_k}$ porque con vectores de dimensión alta los productos punto crecen con varianza d_k y saturarían el softmax (gradientes ≈ 0); (3) el softmax por fila convierte afinidades en pesos que suman 1; (4) la salida de cada token es una media ponderada de los values de *todos* los tokens. Es un diccionario diferenciable de acceso blando: cualquier par de posiciones se conecta a distancia 1, sin importar cuán lejos estén en la secuencia.

🎭 Multi-head, máscara causal y posición

Multi-head: en lugar de una sola atención de dimensión d_model, se ejecutan h atenciones en paralelo sobre proyecciones de dimensión d_k = d_model/h y se concatenan. Cada cabeza puede especializarse (sintaxis, correferencia, posición relativa…).

Máscara causal: en generación (decoder), la posición i solo puede atender a posiciones ≤ i; se implementa poniendo $-\infty$ en las puntuaciones futuras antes del softmax. Esto es lo que permite entrenar la predicción del siguiente token en paralelo sobre toda la secuencia.

Codificación posicional: la atención es invariante al orden (una permutación de los tokens permuta la salida), así que hay que inyectar posición. El paper original usa senos y cosenos de frecuencias geométricas:

```
PE(pos, 2i)   = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

Variantes modernas: embeddings de posición aprendidos, o posiciones relativas/rotatorias.

🧱 El bloque Transformer

```
x ← x + MultiHeadAttention(LayerNorm(x))      (atención + residual)
x ← x + FFN(LayerNorm(x))                     (MLP por token + residual)
```

Cada bloque combina: atención (mezcla información *entre* tokens), una FFN de dos capas aplicada a cada token por separado (procesa la información mezclada), conexiones residuales (clase 053) y layer norm (clase 051). El coste de la atención es **$O(n^2 \cdot d)$** en la longitud n de la secuencia — la razón de toda la investigación en contextos largos.

🧮 Ejemplo trabajado

Atención QKV con 2 tokens y d_k = 2. Supón que las proyecciones ya dieron:

```
Q = | 1 0 |     K = | 1 0 |     V = | 1 2 |
    | 0 1 |         | 0 1 |         | 3 4 |
```

Paso 1 — puntuaciones $Q \cdot K^T/\sqrt{2}$:

```
Q·Kᵀ = | 1 0 |   → escaladas: | 0.7071 0      |
       | 0 1 |                | 0      0.7071 |
```

Paso 2 — softmax por fila (fila 1: e^0.7071 = 2.0281, e^0 = 1):

```
A = | 0.6698 0.3302 |
    | 0.3302 0.6698 |
```

Paso 3 — salida A·V:

```
fila 1: 0.6698·(1,2) + 0.3302·(3,4) = (1.6604, 2.6604)
fila 2: 0.3302·(1,2) + 0.6698·(3,4) = (2.3396, 3.3396)
```

Cada token termina siendo una mezcla de todos los values, dominada por el value del token con key más afín a su query. Con máscara causal, la fila 1 solo vería el token 1: su salida sería exactamente (1, 2).

📊 Propiedades y comparación

Aspecto	RNN/LSTM	CNN 1D	Transformer
Camino máx. entre posiciones	O(n)	O(n/K) por capa	O(1)
Paralelización en secuencia	no	sí	sí
Coste por capa	$O(n \cdot d^2)$	$O(n \cdot K \cdot d^2)$	$O(n^2 \cdot d + n \cdot d^2)$
Sesgo inductivo	orden temporal	localidad	ninguno (posición inyectada)
Contexto práctico	corto-medio	local	largo (limitado por n^2)

⚠️ Errores conceptuales frecuentes

1. **"La atención tiene en cuenta el orden de los tokens."** Es permutation-invariant; sin codificación posicional, "perro muerde hombre" y "hombre muerde perro" serían indistinguibles.
2. **"√d_k es un detalle numérico menor."** Sin el escalado, con d_k grande el softmax satura y los gradientes se anulan: el entrenamiento se degrada de forma medible.
3. **"Cada cabeza de atención tiene un rol interpretable asignado."** Los roles emergen (o no) del entrenamiento; la interpretación por cabezas es un área de investigación, no una garantía de diseño.
4. **"El Transformer procesa los tokens de uno en uno al entrenar."** Entrena en paralelo sobre toda la secuencia gracias a la máscara causal; es en *inferencia* generativa donde produce token a token.
5. **"La FFN del bloque mezcla información entre tokens."** La FFN se aplica a cada posición por separado; la única mezcla entre posiciones ocurre en la atención.

🚀 Del aprendizaje a la operación

De esta mecánica a un LLM en producción median: tokenización BPE, preentrenamiento masivo (clase 059 y parte 05), KV-cache para no recomputar keys/values en generación, atención eficiente (FlashAttention), cuantización para servir con menos memoria, y alineamiento posterior. La matemática de esta clase — softmax($QK^T/\sqrt{d}$)V— es literalmente la que se ejecuta miles de millones de veces por respuesta en un modelo comercial.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("attention")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Vaswani, A. et al. (2017). *Attention Is All You Need*. arXiv:1706.03762 — uso: fuente primaria del mecanismo estudiado
- Bahdanau, D., Cho, K. y Bengio, Y. (2014). *Neural Machine Translation by Jointly Learning to Align and Translate*. arXiv:1409.0473 — uso: fuente primaria del mecanismo estudiado
- Devlin, J. et al. (2018). *BERT: Pre-training of Deep Bidirectional Transformers*. arXiv:1810.04805 — uso: fuente primaria del mecanismo estudiado
- Rush, A. et al. *The Annotated Transformer* (implementación comentada línea a línea). nlp.seas.harvard.edu/annotated-transformer — uso: referencia consultada en su fuente original
- Jurafsky, D. y Martin, J. *Speech and Language Processing* (3.ª ed., borrador), cap. sobre Transformers. web.stanford.edu/~jurafsky/slp3 — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P06 · Aprendizaje de secuencia a secuencia con redes neuronales	2014	Una única red aprende a mapear secuencias de longitud variable a secuencias de longitud variable, de extremo a extremo.	notebook
P07 · Traducción automática neuronal aprendiendo conjuntamente a alinear y traducir	2014	Nace la atención: el decodificador deja de depender de un único vector y consulta toda la entrada en cada paso.	notebook
P08 · La atención es todo lo que necesitas	2017	Elimina la recurrencia y la convolución del modelado de secuencias: todo el cómputo de una capa se paraleliza.	notebook
P20 · Mamba: modelado de secuencias en tiempo lineal con espacios de estados selectivos	2023	El primer competidor serio del Transformer en lenguaje: tiempo lineal y estado de tamaño fijo, sin atención.	notebook
P34 · RoFormer: Transformer mejorado con codificación posicional rotatoria	2021	La posición se codifica rotando, y la atención pasa a depender solo de la distancia relativa. Es la base de casi todo modelo actual.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	citada en las referencias de esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 04 · toda la parte
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 04 · modelos profundos desde la probabilidad

📝 Evaluación completa

❓ Preguntas

1. Define atención y arquitectura transformer sin usar una marca o framework como definición.
2. Explica la relación entre self-attention, multi-head, positional, transformer.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

056 — Graph Neural Networks

Parte: 04 — Redes neuronales y deep learning
Nivel: intermedio-avanzado · **Horas estimadas:** 6
Laboratorio: `neural` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **graph neural networks** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar graph neural networks usando los conceptos `grafos`, `mensajes`, `GCN`, `GAT`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`grafos`, `mensajes`, `GCN`, `GAT`

🗺️ Ubicación en el mapa de la IA

CNN y RNN explotan mallas regulares (píxeles, pasos de tiempo); las GNN generalizan el deep learning a **grafos arbitrarios**: moléculas, redes sociales, sistemas de recomendación, mapas de tráfico, grafos de conocimiento. Su primitiva, el paso de mensajes entre vecinos, conecta el conexionismo con la representación relacional de la parte simbólica del curso, y la atención sobre vecinos (GAT) tiende el puente con la clase 055 — de hecho, un Transformer es una GNN sobre el grafo completo.

📖 Fundamentos

Datos como grafos

Un grafo G = (V, E) tiene nodos con features (X ∈ ℝ^{|V|×d}) y aristas (adyacencia A, opcionalmente con atributos). A diferencia de una imagen, no hay orden canónico de nodos: cualquier arquitectura debe ser **invariante a permutaciones** (renumerar los nodos no puede cambiar el resultado). Tareas típicas: clasificación de nodos (¿fraude?), predicción de aristas (¿recomendar este producto?), clasificación de grafos completos (¿molécula tóxica?).

📬 Paso de mensajes (message passing)

Una capa GNN actualiza cada nodo combinando su estado con un agregado de sus vecinos (Gilmer et al., 2017):

```
m_v = AGGREGATE({ h_u : u ∈ N(v) })       # suma, media o máximo — invariante a orden
h_v' = UPDATE(h_v, m_v)                    # p. ej. σ(W_self·h_v + W_neigh·m_v)
```

Tras k capas, cada nodo ha incorporado información de su vecindario a distancia ≤ k — el análogo del campo receptivo de las CNN. La agregación debe ser invariante a permutaciones (suma/media/máximo), por eso no se usa concatenación ordenada.

🧮 GCN: la convolución sobre grafos

La **Graph Convolutional Network** (Kipf y Welling, 2016) es el caso particular más usado. Con $\hat{A} = A + I$ (añadir self-loops) y $\hat{D}$ su matriz de grados:

```
H' = σ( D̂^(-1/2) · Â · D̂^(-1/2) · H · W )
```

En palabras: cada nodo toma la **media normalizada** de las features propias y de sus vecinos (la normalización simétrica por √grado evita que los nodos muy conectados dominen), la proyecta con una matriz W compartida por todos los nodos y aplica una no linealidad. Es la compartición de pesos de la CNN llevada a vecindarios irregulares.

GAT (Veličković et al., 2017) sustituye la media fija por pesos de atención aprendidos entre vecinos: α_{vu} = softmax de una puntuación calculada con (h_v, h_u). Cada nodo decide cuánto escuchar a cada vecino — atención de la clase 055 restringida al grafo.

🫠 Over-smoothing y profundidad

Apilar muchas capas GNN promedia repetidamente sobre vecindarios: las representaciones de todos los nodos convergen y se vuelven indistinguibles (**over-smoothing**). Por eso las GNN prácticas suelen tener 2-4 capas — o usan residuales, saltos (jumping knowledge) y normalización para ir más allá. Contrasta con CNN de 100 capas: la estructura del grafo mezcla mucho más rápido que una malla.

🧮 Ejemplo trabajado

Grafo línea 1—2—3 con features escalares h = (1, 2, 3), agregación = media con self-loop, sin pesos ni activación (W = identidad):

```
Capa 1:
  h₁' = media(h₁, h₂)      = media(1, 2)    = 1.5
  h₂' = media(h₁, h₂, h₃)  = media(1, 2, 3) = 2.0
  h₃' = media(h₂, h₃)      = media(2, 3)    = 2.5

Capa 2 (sobre 1.5, 2.0, 2.5):
  h₁'' = media(1.5, 2.0)      = 1.75
  h₂'' = media(1.5, 2.0, 2.5) = 2.0
  h₃'' = media(2.0, 2.5)      = 2.25
```

Dos observaciones medibles: (1) tras la capa 1, el nodo 1 ya "sabe" del nodo 2 pero no del 3; tras la capa 2, la información del nodo 3 llegó al 1 (a través del 2): k capas = vecindario a distancia k. (2) El rango de valores se contrajo de [1, 3] a [1.75, 2.25]: el over-smoothing en acción — cada capa de promediado acerca todos los nodos entre sí.

📊 Propiedades y comparación

Aspecto	CNN	GCN	GAT	Transformer
Estructura	malla regular	grafo dado	grafo dado	grafo completo implícito
Pesos por vecino	según posición relativa	iguales (normalizados por grado)	atención aprendida	atención aprendida
Invarianza	traslación	permutación de nodos	permutación de nodos	permutación (sin PE)
Alcance por capa	kernel K×K	vecinos a 1 salto	vecinos a 1 salto	todos los tokens
Riesgo al profundizar	degradación (→ResNet)	over-smoothing	over-smoothing	coste $O(n^2)$

⚠️ Errores conceptuales frecuentes

1. **"Puedo alimentar la matriz de adyacencia a un MLP."** Un MLP sobre A depende de la numeración de los nodos: renumerar cambia la predicción. La invarianza a permutaciones es el requisito que define a las GNN.
2. **"Más capas GNN = mejor, como en visión."** El over-smoothing degrada rápido; 2-4 capas es lo habitual y profundizar exige técnicas específicas.
3. **"La GCN aprende un peso distinto para cada vecino."** La GCN pondera por normalización de grado (fija); pesos por vecino aprendidos es exactamente lo que añade GAT.
4. **"El paso de mensajes puede distinguir cualquier par de grafos."** Su poder está acotado por el test de Weisfeiler-Leman: hay grafos distintos que las GNN de mensajes estándar no separan.
5. **"Un Transformer y una GNN son cosas no relacionadas."** La auto-atención es paso de mensajes sobre el grafo completo con pesos de atención: GAT sin restricción de aristas.

🚀 Del aprendizaje a la operación

En producción (recomendadores, detección de fraude, química) los grafos tienen millones de nodos: hace falta muestreo de vecindarios (GraphSAGE) o entrenamiento por mini-lotes de subgrafos, features heterogéneas por tipo de nodo/arista, y cuidado extremo con la fuga de información entre train/test cuando las aristas mismas son la señal. Librerías como PyTorch Geometric empaquetan estas capas y su muestreo.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("neural")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Kipf, T. y Welling, M. (2016). *Semi-Supervised Classification with Graph Convolutional Networks*. arXiv:1609.02907 — uso: fuente primaria del mecanismo estudiado
- Veličković, P. et al. (2017). *Graph Attention Networks*. arXiv:1710.10903 — uso: fuente primaria del mecanismo estudiado
- Gilmer, J. et al. (2017). *Neural Message Passing for Quantum Chemistry*. arXiv:1704.01212 — uso: fuente primaria del mecanismo estudiado
- Sánchez-Lengeling, B. et al. (2021). *A Gentle Introduction to Graph Neural Networks*. Distill. distill.pub/2021/gnn-intro — uso: referencia consultada en su fuente original
- Documentación de PyTorch Geometric. pytorch-geometric.readthedocs.io — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P120 · Clasificación semisupervisada con redes convolucionales de grafo	2017	Reduce la convolución sobre grafos a una regla de propagación de una línea, y con ella clasifica con una fracción mínima de nodos etiquetados.	notebook
P124 · Redes de atención sobre grafos	2018	Sustituye el promedio uniforme sobre los vecinos por pesos aprendidos por pareja, sin necesitar conocer la estructura global del grafo.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 04 · toda la parte
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 04 · modelos profundos desde la probabilidad

📝 Evaluación completa

❓ Preguntas

1. Define graph neural networks sin usar una marca o framework como definición.
2. Explica la relación entre grafos, mensajes, GCN, GAT.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;

- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

057 — Aprendizaje por refuerzo profundo

Parte: 04 — Redes neuronales y deep learning
Nivel: intermedio-avanzado · **Horas estimadas:** 6
Laboratorio: `robotics` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **aprendizaje por refuerzo profundo** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar aprendizaje por refuerzo profundo usando los conceptos `DQN`, `policy gradient`, `PPO`, `recompensa`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`DQN`, `policy gradient`, `PPO`, `recompensa`

🗺️ Ubicación en el mapa de la IA

El aprendizaje por refuerzo formaliza la decisión secuencial (agente, entorno, recompensa) que el curso trató de forma tabular en la parte 02. El RL *profundo* sustituye las tablas por redes neuronales como aproximadores: DQN (Mnih et al., 2015) jugó Atari desde píxeles, y los policy gradients (PPO) están detrás de la robótica moderna y del RLHF con que se alinean los LLM. Es el punto donde percepción (CNN) y decisión (MDP) se encuentran.

📖 Fundamentos

🎰 El problema: MDP y retorno

Un proceso de decisión de Markov (S, A, P, R, γ) define: estados, acciones, dinámica P(s'|s,a), recompensa r y descuento γ ∈ [0,1). El agente busca una política π(a|s) que maximice el retorno esperado G_t = Σ_k γ^k·r_{t+k}. Dos familias de solución: aprender **valores** (y derivar la política) o aprender la **política** directamente.

🎯 Q-learning y DQN

El valor de acción Q(s,a) = retorno esperado tomando a en s y siguiendo la política óptima después. Q-learning aprende *off-policy* con la actualización:

```
Q(s,a) ← Q(s,a) + α·[ r + γ·max_{a'} Q(s',a') − Q(s,a) ]
                        └──────── objetivo (TD target) ────────┘
```

Con estados continuos o imágenes, la tabla es inviable: **DQN** aproxima Q(s,a;θ) con una red (una CNN si la entrada son píxeles) y minimiza el error TD. Aproximar + bootstrapping + correlación temporal de los datos es una mezcla inestable ("tríada mortal"); DQN la domó con dos trucos que hoy son estándar:

- **Replay buffer**: guardar transiciones (s, a, r, s') y muestrear minibatches aleatorios — rompe la correlación temporal y reutiliza experiencia.
- **Red objetivo (target network)**: calcular el TD target con una copia congelada θ^- que se sincroniza cada N pasos — evita perseguir un objetivo que se mueve con cada actualización.

La exploración se maneja con ε-greedy: con probabilidad ε, acción aleatoria; el resto, argmax Q.

🧭 Policy gradients y actor-critic

En lugar de valores, parametrizar la política π_θ(a|s) y subir por el gradiente del retorno esperado (REINFORCE, Williams 1992):

```
∇J(θ) = E[ Σ_t ∇log π_θ(a_t|s_t) · G_t ]
```

Intuición: aumentar la probabilidad de las acciones que precedieron a retornos altos. El estimador es de alta varianza; restar una **baseline** (el valor V(s), aprendido por un *crítico*) la reduce sin sesgar: A(s,a) = G − V(s) es la **ventaja** y el esquema actor-critic la usa como señal. **PPO** (Schulman et al., 2017) añade un recorte (clip) del ratio π_nueva/π_vieja para impedir pasos de política destructivos, logrando estabilidad con simplicidad — por eso es el caballo de batalla actual (robótica, juegos, RLHF).

⚖️ Valores vs. política

Q-learning explota mejor los datos (off-policy, replay) pero exige acciones discretas y puede sobreestimar valores (de ahí Double DQN). Policy gradients trabajan con acciones continuas y políticas estocásticas, pero son on-policy (datos frescos por actualización) y de mayor varianza.

🧮 Ejemplo trabajado

Una actualización de Q-learning a mano: α = 0.1, γ = 0.9. Estado s con Q(s, derecha) = 0.5. El agente toma "derecha", recibe r = 1 y llega a s' donde max Q(s', ·) = 0.8:

```
objetivo = r + γ·max Q(s',·) = 1 + 0.9·0.8 = 1.72
error TD = 1.72 − 0.5 = 1.22
Q(s, derecha) ← 0.5 + 0.1·1.22 = 0.622
```

Y una de REINFORCE: política softmax sobre 2 acciones con preferencias (θ_izq, θ_der) = (0, 0) → π = (0.5, 0.5). El agente toma "der", obtiene retorno G = +2. Con ∇log π(der) = (−π_izq, 1−π_der) = (−0.5, 0.5) y α = 0.1:

```
θ ← θ + 0.1·2·(−0.5, 0.5) = (−0.1, +0.1)   →   π ≈ (0.45, 0.55)
```

La acción reforzada gana probabilidad; con retorno negativo la habría perdido.

📊 Propiedades y comparación

Aspecto	Q-learning tabular	DQN	REINFORCE	PPO (actor-critic)
Espacio de estados	pequeño y discreto	continuo/imágenes	continuo	continuo
Acciones	discretas	discretas	discretas o continuas	discretas o continuas
Datos	off-policy	off-policy + replay	on-policy	on-policy (con reuso limitado)
Varianza	baja	media	alta	media (ventaja + clip)
Riesgo típico	—	sobreestimación, inestabilidad	varianza, colapso	ajuste fino de hiperparámetros

⚠️ Errores conceptuales frecuentes

1. **"La recompensa define lo que yo quiero; el agente lo entenderá."** El agente optimiza *literalmente* la recompensa: funciones mal especificadas producen trampas (reward hacking) perfectamente racionales.
2. **"Más entrenamiento = curva de recompensa monótona."** El RL profundo es inestable por naturaleza; caídas bruscas tras mejorar son comunes y esperables.
3. **"El replay buffer es solo un caché de eficiencia."** Es una condición de estabilidad: sin romper la correlación temporal, DQN diverge con facilidad.
4. **"REINFORCE con pocas trayectorias basta si el entorno es simple."** El estimador del gradiente tiene varianza enorme; sin baseline ni suficientes muestras, el entrenamiento es ruido.
5. **"Una política que funciona en el simulador funcionará en el mundo real."** La brecha sim-to-real (dinámica, ruido, latencias) degrada políticas aparentemente sólidas; requiere aleatorización de dominio y validación física.

🚀 Del aprendizaje a la operación

Del gridworld al mundo real median: diseño y auditoría de la función de recompensa, simuladores fieles (o datos offline con RL conservador), evaluación con múltiples semillas (la varianza entre corridas es notoria), límites de seguridad duros fuera de la política aprendida, y monitoreo continuo — una política desplegada sigue explorando implícitamente cuando la distribución del entorno cambia.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("robotics")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Sutton, R. y Barto, A. (2018). *Reinforcement Learning: An Introduction* (2.ª ed., PDF oficial gratuito). incompleteideas.net/book/the-book-2nd.html — uso: desarrollo extendido del tema
- Mnih, V. et al. (2015). *Human-level control through deep reinforcement learning* (DQN). Nature 518. doi:10.1038/nature14236 — uso: fuente primaria del mecanismo estudiado
- Mnih, V. et al. (2013). *Playing Atari with Deep Reinforcement Learning*. arXiv:1312.5602 — uso: fuente primaria del mecanismo estudiado
- Schulman, J. et al. (2017). *Proximal Policy Optimization Algorithms*. arXiv:1707.06347 — uso: fuente primaria del mecanismo estudiado
- Williams, R. (1992). *Simple statistical gradient-following algorithms for connectionist reinforcement learning* (REINFORCE). doi:10.1007/BF00992696 — uso: fuente primaria del mecanismo estudiado
- Documentación de Gymnasium (entornos estándar de RL). gymnasium.farama.org — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P26 · Control a nivel humano mediante aprendizaje por refuerzo profundo	2015	El primer agente que aprende a actuar directamente desde píxeles, con la misma arquitectura y los mismos hiperparámetros en decenas de juegos.	notebook
P27 · Dominar el go con redes neuronales profundas y búsqueda en árbol	2016	Une las dos tradiciones de la IA: la búsqueda simbólica de la parte 01 y el aprendizaje profundo de la parte 04, en un solo sistema.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Sutton, Richard S. y Barto, Andrew G. — *Reinforcement Learning: An Introduction*	2.ª · 2018	ISBN 9780262039246 · web de la obra	citada en las referencias de esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 04 · toda la parte
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 04 · modelos profundos desde la probabilidad

📝 Evaluación completa

❓ Preguntas

1. Define aprendizaje por refuerzo profundo sin usar una marca o framework como definición.
2. Explica la relación entre DQN, policy gradient, PPO, recompensa.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

058 — Autoencoders, GAN y difusión

Parte: 04 — Redes neuronales y deep learning
Nivel: intermedio-avanzado · **Horas estimadas:** 6
Laboratorio: `generation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **autoencoders, gan y difusión** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar autoencoders, gan y difusión usando los conceptos `autoencoder`, `GAN`, `difusión`, `latente`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`autoencoder`, `GAN`, `difusión`, `latente`

🗺️ Ubicación en el mapa de la IA

Hasta aquí las redes *discriminaban* (clasificar, predecir); los modelos generativos aprenden la **distribución** de los datos para producir muestras nuevas. Tres familias marcan la evolución: autoencoders y VAE (2013, espacios latentes probabilísticos), GAN (2014, el juego adversario que dominó la síntesis de imágenes una década) y difusión (2020, el estado del arte detrás de Stable Diffusion, DALL·E y la generación de audio y video actual). Sus espacios latentes conectan con los embeddings de la parte 05.

📖 Fundamentos

🗜️ Autoencoders y VAE

Un **autoencoder** comprime y reconstruye: encoder $z = f(x)$ con $\dim(z) \ll \dim(x)$, decoder $\hat{x} = g(z)$, pérdida $\|x - \hat{x}\|^2$. El cuello de botella obliga a aprender la estructura esencial de los datos (un autoencoder lineal recupera el subespacio de PCA). Usos: reducción de dimensión, denoising, detección de anomalías (error de reconstrucción alto = dato atípico). Pero su espacio latente no es muestreable: puntos intermedios pueden decodificar basura.

El **autoencoder variacional** (VAE; Kingma y Welling, 2013) lo arregla haciendo el latente probabilístico: el encoder produce μ(x) y σ(x), se muestrea z = μ + σ·ε con ε ~ N(0,1) (el **truco de reparametrización**, que deja pasar el gradiente a través del muestreo) y se optimiza el ELBO:

```
L = E[log p(x|z)]  −  KL( q(z|x) ‖ N(0, I) )
    reconstrucción     regularización del latente
```

El término KL empaqueta el latente alrededor de una normal estándar: muestrear z ~ N(0,I) y decodificar produce datos nuevos coherentes.

⚔️ GAN: el juego adversario

Una **GAN** (Goodfellow et al., 2014) enfrenta dos redes: el generador G transforma ruido z en muestras; el discriminador D intenta distinguir muestras reales de generadas:

```
min_G max_D  E_x[log D(x)] + E_z[log(1 − D(G(z)))]
```

En el equilibrio teórico, la distribución de G iguala a la de los datos y D no puede hacer mejor que 50 %. En la práctica el entrenamiento es delicado: **mode collapse** (G produce poca variedad), oscilaciones y gradientes que se desvanecen si D domina. Las GAN producen muestras nítidas y rápidas (una pasada), pero sin verosimilitud explícita ni cobertura garantizada de la distribución.

🌫️ Modelos de difusión

La difusión (DDPM; Ho et al., 2020) genera invirtiendo un proceso de ruido. **Forward** (fijo, sin aprendizaje): se añade ruido gaussiano en T pasos hasta destruir la señal; en forma cerrada, con $\bar{\alpha}_t$ el producto acumulado de (1−β):

```
x_t = √ᾱ_t · x₀ + √(1−ᾱ_t) · ε,      ε ~ N(0, I)
```

Reverse (aprendido): una red $\varepsilon_\theta(x_t, t)$ predice el ruido añadido; la pérdida es simplemente $\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$. Generar = partir de ruido puro x_T y denoisear paso a paso. Entrenamiento estable (es regresión, no un juego), cobertura excelente de la distribución y calidad estado del arte; el precio es la generación iterativa (decenas o cientos de pasadas de red frente a una de la GAN). Los modelos texto-imagen añaden condicionamiento (el prompt guía cada paso de denoising) y suelen difundir en el latente de un autoencoder (latent diffusion) para abaratar.

🧮 Ejemplo trabajado

Autoencoder lineal 2→1→2 a mano. Encoder z = w·x con w = (0.6, 0.8) (norma 1); decoder $\hat{x}$ = z·w.

```
x = (3, 4):   z = 0.6·3 + 0.8·4 = 5.0   →  x̂ = (3.0, 4.0)   error = 0
x = (4, 3):   z = 2.4 + 2.4 = 4.8       →  x̂ = (2.88, 3.84) error² = 1.12²+0.84² = 1.96
```

El punto alineado con la dirección w se reconstruye perfecto; el resto pierde su componente ortogonal: comprimir a 1D = proyectar sobre la mejor recta (PCA).

Forward de difusión a mano. $x_0 = 1.0$, $\varepsilon = 0.2$:

```
ᾱ = 0.9:  x_t = √0.9·1 + √0.1·0.2 = 0.9487 + 0.0632 = 1.0119   (casi señal)
ᾱ = 0.5:  x_t = 0.7071 + 0.1414 = 0.8485                        (mitad y mitad)
ᾱ = 0.1:  x_t = 0.3162 + 0.1897 = 0.5060                        (domina el ruido)
```

La red de denoising ve x_t y t, y debe recuperar ε: en $\bar{\alpha}$ alto es fácil, en $\bar{\alpha}$ bajo casi todo es ruido — por eso el schedule de ruido es una decisión de diseño central.

📊 Propiedades y comparación

Aspecto	Autoencoder/VAE	GAN	Difusión
Objetivo	reconstrucción (+KL en VAE)	juego minimax	regresión de ruido
Estabilidad de entrenamiento	alta	baja (collapse, oscilación)	alta
Calidad de muestra	media (VAE borroso)	alta	la más alta
Velocidad de generación	1 pasada	1 pasada	T pasadas (iterativa)
Cobertura de la distribución	buena	riesgo de mode collapse	muy buena
Verosimilitud	cota (ELBO)	no disponible	cota estimable

⚠️ Errores conceptuales frecuentes

1. **"Un autoencoder genera datos nuevos muestreando su latente."** Sin el término KL del VAE, el latente tiene huecos: decodificar puntos no vistos produce artefactos.
2. **"La pérdida del generador GAN baja = va mejorando."** Las pérdidas adversarias son relativas al oponente; la calidad se evalúa con métricas externas (FID) e inspección, no con la curva de pérdida.
3. **"La difusión aprende a añadir ruido."** El forward es fijo y sin parámetros; lo aprendido es el *reverse* (predecir el ruido para quitarlo).
4. **"El VAE es borroso porque le falta entrenamiento."** Es estructural: la reconstrucción en media bajo un decoder gaussiano promedia detalles; por eso las variantes perceptuales y los modelos latentes.
5. **"Mode collapse significa que la GAN no aprende nada."** Aprende — pero concentra su masa en pocos modos convincentes; las muestras son buenas y la *diversidad* es la que está rota.

🚀 Del aprendizaje a la operación

Producción generativa implica: métricas de calidad y diversidad (FID, precision/recall de distribuciones), control del condicionamiento (prompts, clases), filtros de seguridad sobre lo generado, trazabilidad de datos de entrenamiento (derechos y sesgos) y costes de inferencia (la difusión multiplica el cómputo por sus pasos; la destilación de pasos — clase 059 en espíritu — los reduce a 1-4).

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("generation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Kingma, D. y Welling, M. (2013). *Auto-Encoding Variational Bayes* (VAE). arXiv:1312.6114 — uso: fuente primaria del mecanismo estudiado
- Goodfellow, I. et al. (2014). *Generative Adversarial Networks*. arXiv:1406.2661 — uso: fuente primaria del mecanismo estudiado
- Ho, J., Jain, A. y Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. arXiv:2006.11239 — uso: fuente primaria del mecanismo estudiado
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*, cap. 14 (Autoencoders). deeplearningbook.org/contents/autoencoders.html — uso: desarrollo extendido del tema
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*, cap. 20 (Deep Generative Models). deeplearningbook.org/contents/generative_models.html — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P38 · Bayes variacional con autocodificación	2013	Hace entrenable un modelo generativo latente: el truco de reparametrización deja pasar el gradiente a través del muestreo.	notebook
P39 · Redes generativas adversarias	2014	Convierte la generación en un juego: dos redes compiten y ninguna necesita una verosimilitud explícita.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 14 · obra de referencia de la parte 04
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 04 · modelos profundos desde la probabilidad

📝 Evaluación completa

❓ Preguntas

1. Define autoencoders, gan y difusión sin usar una marca o framework como definición.
2. Explica la relación entre autoencoder, GAN, difusión, latente.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

059 — Transferencia, fine-tuning y destilación

Parte: 04 — Redes neuronales y deep learning
Nivel: intermedio-avanzado · **Horas estimadas:** 6
Laboratorio: `neural` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **transferencia, fine-tuning y destilación** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar transferencia, fine-tuning y destilación usando los conceptos `transfer`, `fine-tuning`, `distillation`, `PEFT`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`transfer`, `fine-tuning`, `distillation`, `PEFT`

🗺️ Ubicación en el mapa de la IA

Entrenar desde cero exige datos y cómputo que casi nadie tiene; la transferencia convirtió el deep learning en tecnología de uso general: un modelo preentrenado (ImageNet, un LLM) se adapta a cada tarea con una fracción del coste. Fine-tuning, PEFT/LoRA y destilación son hoy el modo *normal* de construir sistemas — incluida la especialización de LLM de la parte 05 — y el puente directo entre esta parte y la IA aplicada del resto del curso.

📖 Fundamentos

🔁 Transferencia de representaciones

Las primeras capas de una red aprenden features genéricas (bordes y texturas en visión; morfología y sintaxis en lenguaje) y las últimas, features específicas de la tarea (Yosinski et al., 2014). Eso habilita dos estrategias sobre un modelo preentrenado:

- **Feature extraction**: congelar el tronco y entrenar solo una cabeza nueva. Barato, robusto con pocos datos, pero no adapta las representaciones.

- **Fine-tuning**: reentrenar todo (o las capas superiores) con tasa de aprendizaje baja (típicamente 10-100× menor que la del preentrenamiento). Más potente, pero con pocos datos puede sobreajustar y sufrir **olvido catastrófico** (perder lo aprendido en preentrenamiento).

Regla práctica: cuanto más pequeño el dataset y más parecido al dominio original, más congelar; cuanto más grande y distinto, más descongelar.

🪶 PEFT y LoRA

El fine-tuning completo de un modelo de miles de millones de parámetros multiplica memoria (gradientes + estados de Adam por parámetro) y obliga a guardar una copia entera por tarea. El *parameter-efficient fine-tuning* entrena solo una fracción. **LoRA** (Hu et al., 2021) congela cada matriz $W \in \mathbb{R}^{d\times k}$ y aprende una corrección de rango bajo:

$$W' = W + \Delta W = W + B\cdot A \qquad \text{con } B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d, k)$$

Parámetros entrenables: $r\cdot(d+k)$ en lugar de $d\cdot k$. Con $r = 8$ sobre una matriz 768×768: 12 288 frente a 589 824 (≈ 2 %). Ventajas operativas: los adaptadores se almacenan por tarea en megabytes, se intercambian sin tocar el modelo base y ΔW puede fusionarse en W para inferencia sin sobrecoste.

🧪 Destilación de conocimiento

La **destilación** (Hinton et al., 2015) comprime un modelo grande (teacher) en uno pequeño (student) entrenando al student para imitar las *probabilidades* del teacher, no solo la etiqueta dura. Con temperatura T en el softmax:

$$p_i(T) = \exp(z_i/T) \,/\, \Sigma_j \exp(z_j/T)$$
$$L = \lambda\cdot CE(y, \text{student}) + (1-\lambda)\cdot T^2\cdot KL(\ \text{teacher}(T)\ \|\ \text{student}(T)\ )$$

$T > 1$ suaviza la distribución y revela el "conocimiento oscuro": qué clases se parecen según el teacher (un 7 se parece más a un 1 que a un 8). Esa señal por clase es mucho más rica que la etiqueta y permite al student pequeño acercarse al rendimiento del teacher (ejemplo canónico: DistilBERT ≈ 97 % de BERT con 40 % menos parámetros).

🧮 Ejemplo trabajado

Conteo LoRA. Capa de atención con $W_Q, W_K, W_V, W_O \in \mathbb{R}^{768\times768}$ y $r = 8$:

$$\text{Full fine-tuning: } 4\cdot768\cdot768 = 2\,359\,296 \text{ parámetros}$$
$$\text{LoRA: } 4\cdot8\cdot(768+768) = 49\,152 \text{ parámetros } (2.08\ \%)$$

Softmax con temperatura a mano. Logits del teacher $z = (3, 1)$ para 2 clases:

$$T = 1:\ p = (e^3, e^1)/(e^3+e^1) = (20.09, 2.72)/22.81 = (0.881, 0.119)$$
$$T = 4:\ z/T = (0.75, 0.25) \to (2.117, 1.284)/3.401 = (0.622, 0.378)$$

Con T = 1 el student casi solo ve "clase 1"; con T = 4 aprende además *cuánto* se parece la entrada a la clase 2 — la estructura de similitud que la etiqueta dura descarta. El factor T^2 de la pérdida compensa que los gradientes se encogen $\propto 1/T^2$.

📊 Propiedades y comparación

Estrategia	Parámetros entrenados	Datos necesarios	Riesgo principal	Artefacto resultante
Feature extraction	solo la cabeza (≪1 %)	decenas-miles	features no adaptadas	cabeza pequeña
Fine-tuning completo	100 %	miles-millones	olvido catastrófico, coste	copia entera del modelo
LoRA / PEFT	0.1-2 %	miles	expresividad limitada por r	adaptador de MB
Destilación	100 % del student	sin etiquetar sirve (usa al teacher)	techo = teacher	modelo pequeño independiente

⚠️ Errores conceptuales frecuentes

1. **"Fine-tuning con la misma η del preentrenamiento."** Destroza las representaciones en las primeras iteraciones; la η debe ser 1-2 órdenes menor.
2. **"Transferir siempre ayuda."** Con dominios muy lejanos (imágenes naturales → señales médicas crudas) la transferencia puede ser neutra o negativa; hay que medirla contra un baseline desde cero.
3. **"LoRA aproxima peor porque entrena menos parámetros, así que siempre rinde peor."** En adaptación de tareas, los ΔW efectivos suelen tener rango bajo intrínseco; LoRA iguala al fine-tuning completo en muchos benchmarks.
4. **"La destilación es solo entrenar con las predicciones top-1 del teacher."** La señal útil está en la distribución *completa* suavizada por temperatura; con etiquetas duras del teacher se pierde la mayor parte del beneficio.
5. **"Congelar capas evita todo sobreajuste."** Reduce capacidad de sobreajuste del tronco, pero la cabeza puede sobreajustar igual; la validación sigue siendo obligatoria.

🚀 Del aprendizaje a la operación

El flujo industrial típico: elegir modelo base (licencia incluida), fine-tuning con PEFT sobre datos propios versionados, evaluación contra el modelo base y un baseline simple, destilación si la latencia o el coste de servir lo exigen, y registro de qué pesos/adaptadores produjeron qué métricas (la trazabilidad de la clase 060). El riesgo operativo nuevo: heredar sesgos y vulnerabilidades del modelo base sin haberlos medido.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("neural")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Hinton, G., Vinyals, O. y Dean, J. (2015). *Distilling the Knowledge in a Neural Network*. arXiv:1503.02531 — uso: fuente primaria del mecanismo estudiado
- Yosinski, J. et al. (2014). *How transferable are features in deep neural networks?* arXiv:1411.1792 — uso: fuente primaria del mecanismo estudiado
- Hu, E. et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685 — uso: fuente primaria del mecanismo estudiado
- Howard, J. y Ruder, S. (2018). *Universal Language Model Fine-tuning for Text Classification* (ULMFiT). arXiv:1801.06146 — uso: fuente primaria del mecanismo estudiado
- Tutorial oficial de PyTorch: Transfer Learning for Computer Vision — uso: referencia consultada en su fuente original
- Documentación de Hugging Face PEFT. huggingface.co/docs/peft — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P45 · Destilar el conocimiento de una red neuronal	2015	Las probabilidades del maestro contienen más información que la etiqueta correcta: el modelo pequeño aprende de esa estructura.	notebook
P48 · LoRA: adaptación de rango bajo de modelos de lenguaje grandes	2021	Ajustar un modelo enorme entrenando una fracción diminuta de parámetros, sin coste añadido en inferencia.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 04 · toda la parte
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 04 · modelos profundos desde la probabilidad

📝 Evaluación completa

❓ Preguntas

1. Define transferencia, fine-tuning y destilación sin usar una marca o framework como definición.
2. Explica la relación entre transfer, fine-tuning, distillation, PEFT.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;

- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

060 — Proyecto: modelo trazable de extremo a extremo

Parte: 04 — Redes neuronales y deep learning
Nivel: intermedio-avanzado · **Horas estimadas:** 10
Laboratorio: `capstone` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **proyecto: modelo trazable de extremo a extremo** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: modelo trazable de extremo a extremo usando los conceptos `dataset card`, `model card`, `registro`, `despliegue`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`dataset card`, `model card`, `registro`, `despliegue`

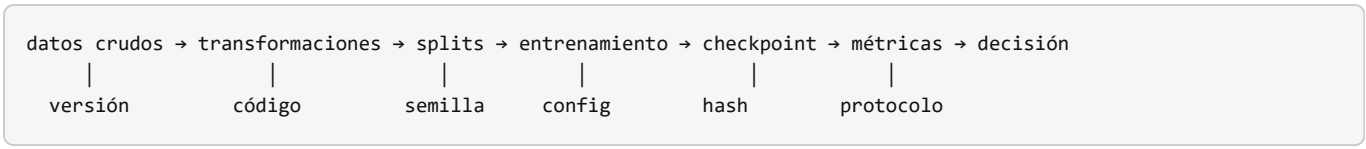
🗺️ Ubicación en el mapa de la IA

Cierre de la parte 04: todo lo anterior (arquitecturas, optimización, transferencia) solo tiene valor si el resultado es **verificable por terceros**. La crisis de reproducibilidad en ML (resultados que no se replican por semillas, datos o código no declarados) produjo un estándar emergente — dataset cards, model cards, registro de experimentos — que este proyecto practica de extremo a extremo y que la parte 05 asumirá como requisito al trabajar con LLM.

📖 Fundamentos

📑 Trazabilidad: la cadena completa

Un modelo es trazable cuando cada afirmación sobre él puede seguirse hasta su origen:



Si un eslabón no está registrado, la cadena se rompe: una métrica sin split declarado, o un checkpoint sin configuración, no son evidencia sino anécdota.

🖨️ Dataset cards y model cards

Datasheets for Datasets (Gebru et al., 2018): documento que responde, para un dataset: motivación, composición, proceso de recolección, preprocesamiento, usos recomendados y desaconsejados, sesgos conocidos. **Model Cards** (Mitchell et al., 2019): lo análogo para modelos — uso previsto, datos de entrenamiento, métricas **desagregadas por subgrupos relevantes**, condiciones de fallo conocidas y consideraciones éticas. Ambos son hoy práctica estándar (los hubs de modelos los integran) y la base de la documentación exigida por marcos regulatorios.

🔁 Reproducibilidad técnica

Fuentes de no determinismo y su control:

- **Semillas**: fijar las de *todas* las librerías (random, NumPy, framework) y registrarlas como parte del experimento.
- **No determinismo de GPU**: algunas operaciones CUDA son no deterministas por rendimiento; los frameworks ofrecen modos deterministas (más lentos) — la reproducibilidad bit a bit tiene coste y a veces se sustituye por reproducibilidad estadística (misma media ± varianza entre semillas).
- **Entorno**: versiones de librerías y drivers fijadas (lockfiles, contenedores).
- **Datos**: hash del dataset y de cada split; el *data leakage* entre train y test invalida silenciosamente todo lo demás.

📊 Protocolo de evaluación honesto

Decidir **antes** de mirar el test set: métrica principal (y por qué, dado el costo de cada tipo de error), baseline contra el que comparar (mayoría, regla simple, modelo anterior), splits (y si hay grupos —pacientes, usuarios — splits por grupo), y número de corridas con semillas distintas para reportar media ± desviación. El test set se toca una vez. Reportar también métricas desagregadas: un accuracy global puede esconder un fallo grave en un subgrupo.

🧮 Ejemplo trabajado

Evaluación trazable de un clasificador binario. Matriz de confusión sobre el test (tocado una única vez, split con semilla registrada 60):

```
               predicho +   predicho −
real +            TP=40        FN=20
real −            FP=10        TN=30

precisión = 40/(40+10) = 0.800
recall    = 40/(40+20) = 0.667
F1        = 2·(0.8·0.667)/(0.8+0.667) = 0.727
accuracy  = 70/100 = 0.700
baseline mayoritario (predecir siempre +): accuracy = 60/100 = 0.600
```

Lectura honesta: el modelo supera al baseline en accuracy (0.70 vs 0.60), pero deja escapar un tercio de los positivos (recall 0.667). Si el costo de un falso negativo es alto (p. ej. detección de fallos), esa cifra —no el accuracy— es la que decide, y así debe constar en la model card junto con: semilla, hash de los splits, configuración de entrenamiento y desviación entre corridas (p. ej. accuracy 0.70 ± 0.02 sobre 5 semillas). Con eso, un tercero puede reproducir y auditar la afirmación.

📊 Propiedades y comparación

Nivel de trazabilidad	Qué registra	Qué permite	Qué falla sin él
Código versionado	commit del entrenamiento	re-ejecutar	"funcionaba en mi máquina"
Config + semillas	hiperparámetros, seeds	reproducir la corrida	métricas irrepetibles
Datos versionados	hash de dataset y splits	auditar leakage	evidencia contaminada
Registro de experimentos	corridas, métricas, artefactos	comparar honestamente	cherry-picking involuntario
Cards (datos + modelo)	usos, límites, sesgos	decisión informada de terceros	despliegue a ciegas

⚠️ Errores conceptuales frecuentes

1. **"Fijé la semilla, luego es reproducible."** La semilla es un eslabón; sin versiones de entorno, hash de datos y config registrada, la cadena sigue rota.
2. **"Elegí la mejor corrida de 20 para el informe."** Eso es selección post-hoc: la estimación honesta es media ± desviación de todas, o una corrida nueva con protocolo prefijado.
3. **"El accuracy global resume el modelo."** Puede ocultar fallos por subgrupo y es engañoso con clases desbalanceadas (el baseline mayoritario ya da el 60 % aquí).
4. **"Validar varias veces contra el test set está bien si no entreno con él."** Cada mirada al test para tomar decisiones filtra información: es sobreajuste de selección, más lento pero igual de real.

5. **"Las cards son burocracia."** Son el contrato que permite a un tercero decidir si el modelo sirve para *su* caso; sin ellas, cada reutilización es un experimento a ciegas.

🚀 Del aprendizaje a la operación

En producción se añaden: registro de modelos con promoción por etapas (staging → producción), monitoreo de deriva de datos y de rendimiento en vivo, rollback automático ante degradación, auditoría de acceso a datos sensibles y revalidación periódica. La disciplina es la misma de esta clase; solo cambia la escala y que el "tercero que audita" puede ser un regulador.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("capstone")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Mitchell, M. et al. (2019). *Model Cards for Model Reporting*. arXiv:1810.03993 — uso: fuente primaria del mecanismo estudiado
- Gebru, T. et al. (2018). *Datasheets for Datasets*. arXiv:1803.09010 — uso: fuente primaria del mecanismo estudiado
- Pineau, J. et al. (2020). *Improving Reproducibility in Machine Learning Research*. arXiv:2003.12206 — uso: fuente primaria del mecanismo estudiado
- Notas oficiales de PyTorch sobre reproducibilidad y determinismo. pytorch.org/docs/stable/notes/randomness.html — uso: referencia consultada en su fuente original
- Documentación de MLflow (registro de experimentos y modelos). mlflow.org/docs/latest — uso: referencia consultada en su fuente original

---

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 04 · toda la parte
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	obra de referencia de la parte 04 · modelos profundos desde la probabilidad

📝 Evaluación completa

❓ Preguntas

1. Define proyecto: modelo trazable de extremo a extremo sin usar una marca o framework como definición.
2. Explica la relación entre dataset card, model card, registro, despliegue.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.
